

Viajes Aventura

Informe Técnico “Desarrollo de Software”

Asignatura: Programación Orientada a Objeto Seguro

Caso de Estudio: Caso “Viajes Aventura”

Sección: V-IEI-N2-P3-C1

Nombre del académico: Rubén Alejandro Schnettler Lucero

Nombre estudiantes: Benjamín Comanato (Líder de Proyecto)

Sergio Antonio Ruiz Espinoza

Bernardo Fuentes Ibaceta

Sede: INACAP, Valparaíso

Fecha de entrega: 05 de octubre del 2026

INDICE

- 1. Introducción 4
 - 1.1 Contexto..... 4
 - 1.2 Objetivos 4
 - 1.3 Alcance 4
 - 1.4 Enfoque de trabajo 5
- 2. Levantamiento de requerimientos (criterio 4.1.1) 5
 - 2.1 Análisis de la problemática 5
 - 2.2 De reglas de negocio a requerimientos 6
 - 2.3 Requerimientos funcionales 6
 - 2.4 Requerimientos no funcionales 7
 - 2.5 Clasificación, priorización y verificación..... 9
 - 2.6 Supuestos por vacíos del caso..... 10
- 3. Modelado de la solución (criterio 4.1.2)..... 11
 - 3.1 Diagrama de casos de uso 11
 - 3.2 Modelos de procesos de negocio (BPMN)..... 12
 - 3.3 Diagramas de clases UML 14
 - 3.3.1 Clases de dominio 14
 - 3.3.2 Clases de servicios, acceso a datos y seguridad 15
 - 3.4 Modelo de datos..... 16
 - 3.5 Arquitectura de la solución 17
 - 3.6 Trazabilidad entre requerimientos y modelos (Indicador 4.1.2.I.8)..... 18
- 4. Planificación con metodologías ágiles (criterio 4.1.3)..... 21
 - 4.1 Marco de trabajo y roles 21
 - 4.2 Product Backlog priorizado..... 22
 - 4.3 Sprint Backlog por iteración..... 23
 - 4.4 Tiempos, entregables y riesgos..... 24
- 5. Implementación en Python con persistencia (criterio 4.1.4) 25
 - 5.1 Tecnologías y organización del código..... 25
 - 5.2 Programación orientada a objetos aplicada..... 26
 - 5.3 Persistencia de datos 26
 - 5.4 Funcionalidades CRUD 27
 - 5.5 Pruebas y verificación 27
 - 5.6 Limitaciones conocidas 30
- 6. Acceso seguro a la solución (criterio 4.1.5)..... 31
 - 6.1 Librerías de repositorios oficiales y criterios de calidad 31
 - 6.2 Mecanismo de autenticación 32
 - 6.3 Validación rigurosa de credenciales (Indicador 4.1.5.G.18)..... 32

6.4 Protección de la información sensible (Indicador 4.1.5.I.19).....	33
6.5 Análisis de amenazas y controles	33
6.6 Evaluación y optimización de la seguridad con apoyo de IA (Indicador 4.1.5.I.20)	34
6.7 Riesgos residuales y recomendaciones	35
7. Uso crítico de herramientas de inteligencia artificial.....	35
7.1 Proceso de revisión aplicado.....	35
7.2 Registro de contribuciones, análisis y decisiones	35
7.3 Riesgos del uso de IA y cómo se controlaron.....	36
8. Conclusiones	37
Referencias	37

1. Introducción

1.1 Contexto

Viajes Aventura es una agencia de Valparaíso creada por tres socios que arma y vende paquetes turísticos combinando destinos que ellos mismos han recorrido. Hoy opera sin sistema: el catálogo vive en una planilla compartida, las reservas en un cuaderno y los datos de los clientes (nombre, RUT, teléfono y correo) en el cuaderno, en la planilla y en las conversaciones de mensajería de cada socio. En la última temporada registraron 214 reservas, de las cuales 19 estaban duplicadas, 6 superaron el cupo del paquete, 3 se tomaron sobre paquetes con fecha de salida vencida y 4 paquetes se publicaron con un precio distinto del cobrado.

Este informe documenta la solución completa: el levantamiento de requerimientos, el modelado con UML y BPMN, la planificación ágil, la implementación en Python con persistencia de datos y el acceso seguro, junto con el uso crítico de herramientas de inteligencia artificial durante el desarrollo.

1.2 Objetivos

Objetivo general. Desarrollar un sistema en Python que permita a Viajes Aventura gestionar destinos, paquetes turísticos y reservas de clientes, garantizando la integridad de la información y la protección de las credenciales y de los datos sensibles.

- Identificar, clasificar, priorizar y validar los requerimientos funcionales y no funcionales a partir del caso (criterio 4.1.1).
- Modelar estructura y procesos con casos de uso, BPMN y diagramas de clases UML, asegurando trazabilidad con los requerimientos (4.1.2).
- Planificar el trabajo con Scrum: roles, Product Backlog, Sprint Backlog, tiempos y entregables (4.1.3).
- Implementar la solución aplicando programación orientada a objetos, persistencia en base de datos y CRUD, evaluando críticamente el código sugerido por IA (4.1.4).
- Implementar autenticación y protección de datos con librerías de repositorios oficiales y evaluar la seguridad del sistema (4.1.5).

1.3 Alcance

La Tabla 1 delimita lo que incluye la primera versión del sistema y lo que queda fuera de ella.

Tabla 1. Alcance de la versión 1: dentro y fuera del alcance.

Dentro del alcance (versión 1)	Fuera del alcance (acordado por los socios)
Destinos: registrar, modificar, dejar no disponible y listar. Paquetes: crear combinando destinos, definir fechas y cupo, calcular el precio y consultar disponibilidad. Reservas: registro y autenticación de clientes, reservar, almacenar y consultar el historial propio. Seguridad: autenticación, validación de todo dato ingresado y protección de credenciales y datos sensibles.	Pasarela de pago y verificación automática de transferencias · facturación electrónica ante el SII · aplicación para teléfonos · integración con aerolíneas, hoteles u operadores · envío de correos o mensajería al cliente · informes de gestión · contabilidad y remuneraciones.

El caso declara explícitamente que contiene vacíos (qué ocurre cuando un cliente desiste, cómo se comporta un paquete al terminar su temporada, quién modifica el catálogo). Cada vacío se resolvió con un supuesto declarado y fundamentado técnicamente en la sección 2.6.

1.4 Enfoque de trabajo

El trabajo se organizó con Scrum, en iteraciones cortas con entregas revisables; el detalle de roles, backlog e iteraciones se presenta en la sección 4. La solución se construyó en capas (presentación, servicios, dominio, acceso a datos y seguridad), con dos interfaces que comparten la misma lógica de negocio: una de consola y una web. Cada regla de negocio del caso se tradujo a un requerimiento, se modeló, se implementó y se verificó mediante pruebas automatizadas; la trazabilidad completa se presenta en la sección 3.6. Las decisiones técnicas y sus alternativas descartadas se fundamentan en las secciones 5 y 6.

2. Levantamiento de requerimientos (criterio 4.1.1)

2.1 Análisis de la problemática

Las cifras y testimonios del caso se analizaron para separar el síntoma observado de su causa raíz. Esto evita proponer requerimientos que solo “parchen” el síntoma (por ejemplo, agregar una columna a la planilla) y orienta al sistema a eliminar la causa. El análisis se resume en la Tabla 2.

Tabla 2. Evidencia del caso, causa raíz y respuesta del sistema.

Evidencia del caso	Causa raíz identificada	Respuesta del sistema (req.)
19 reservas duplicadas (misma reserva escrita por dos socios)	No existe un registro único ni identificación del cliente; cada socio anota por su cuenta	Reserva ligada a un cliente autenticado; una sola reserva activa por cliente y paquete (RF-11, RF-18)
6 reservas por sobre el cupo	El cupo “se lleva en la cabeza”; no hay un dato de cupo disponible que el sistema compare	Cupo disponible calculado = cupo máximo – personas reservadas, verificado en transacción (RF-11, RNF-05)
3 reservas sobre fechas vencidas	Nadie compara contra la fecha del día; los paquetes vencidos siguen visibles	Rechazo de reservas con fecha de salida pasada; el cliente solo ve paquetes vigentes (RF-08, RF-11)
4 paquetes con precio distinto del cobrado	El costo de un destino cambia después de vendido el paquete y la planilla recalcula	Precio congelado al publicar; total de la reserva fijo (RF-07, RF-11)
Error de un cero al armar un paquete (publicado a la décima parte)	Cálculo manual: copiar costos, sumar y aplicar margen a mano	Cálculo automático del precio = suma de costos + margen, con margen validado (RF-06)
4 destinos sin operar aún visibles; no se borran porque están en paquetes vendidos	No hay baja lógica; borrar rompería el contenido de paquetes ya vendidos	Eliminar si está libre; si está en algún paquete, marcar NO DISPONIBLE (RF-03, RF-04)
31 consultas respondidas revisando el cuaderno hoja por hoja; no se sabe cuántas personas han viajado	Datos sin estructura ni consulta	Base de datos persistente e historial de reservas por cliente (RF-12)
RUT y teléfono en una planilla enviada por correo; riesgo de pérdida	Datos sensibles en texto plano, sin control de acceso	Cifrado en reposo, contraseñas con hash, acceso por rol y ocultamiento en listados (RF-13, RF-15, RNF-01, RNF-02)
Destinos repetidos con nombres distintos; columnas agregadas por cada socio	Sin validación ni restricciones de integridad	Nombre de destino único y validación de todo dato de entrada (RF-01, RF-14)

2.2 De reglas de negocio a requerimientos

El caso entrega 17 reglas de negocio (R1 a R17) y advierte que una regla describe cómo funciona el negocio, mientras que un requerimiento describe qué debe hacer el sistema. Por eso cada regla se tradujo a uno o más requerimientos verificables, redactados con la estructura “El sistema deberá...” y acompañados de un criterio de aceptación medible. Ninguna regla quedó sin cobertura (véase la matriz de cobertura de la sección 2.5).

2.3 Requerimientos funcionales

Prioridad según MoSCoW: M = Debe (imprescindible), S = Debería (aporta valor alto, no bloquea), C = Podría, W = No se hará en esta versión. Origen: regla de negocio (R#), entrevista o supuesto (S#). Los requerimientos funcionales se listan en la Tabla 3.

Tabla 3. Requerimientos funcionales (RF).

ID	Requerimiento	Prior.	Origen	Criterio de aceptación
RF-01	El sistema deberá permitir al administrador registrar un destino con nombre, zona, descripción, duración en días y costo base por persona.	M	R1, R2	Se rechaza un nombre repetido (sin distinguir mayúsculas) y un costo ≤ 0 .
RF-02	El sistema deberá permitir al administrador modificar los datos de un destino existente.	M	R1, R2, R7	Los datos se actualizan y los paquetes ya publicados conservan su precio.
RF-03	El sistema deberá eliminar un destino que no pertenece a ningún paquete y, si pertenece a alguno, marcarlo como no disponible sin eliminarlo.	M	R8	Destino libre desaparece; destino en paquete queda con disponible = No y el paquete vendido conserva su contenido.
RF-04	El sistema deberá listar los destinos y ofrecer para paquetes nuevos solo los disponibles.	M	R8	El listado muestra el estado; un destino no disponible no puede agregarse a un paquete.
RF-05	El sistema deberá permitir crear un paquete con nombre, fecha de salida, fecha de regreso, cupo máximo y entre 2 y 5 destinos distintos.	M	R3, R4, R5	Se rechazan < 2 o > 5 destinos, destinos repetidos, regreso $\leq$ salida y cupo ≤ 0 ; un destino puede estar en varios paquetes.
RF-06	El sistema deberá calcular el precio por persona como la suma de los costos base de los destinos más el margen de operación (20 % por defecto, nunca negativo).	M	R6	Destinos 120.000 + 310.000 con margen 20 % $\rightarrow$ 516.000; un margen negativo se rechaza.
RF-07	El sistema deberá fijar el precio al publicar el paquete, sin alterarlo si el costo de un destino cambia después.	M	R7	Tras modificar el costo de un destino, el paquete publicado mantiene su precio.
RF-08	El sistema deberá permitir consultar los paquetes vigentes con su precio, destinos y cupo disponible.	M	R14, R15	El cliente no ve paquetes con salida vencida; el cupo mostrado = máximo – reservado.
RF-09	El sistema deberá permitir que un cliente se registre con nombre, RUT, correo, teléfono y contraseña.	M	R9, R10	Correo único; RUT válido (módulo 11); contraseña que cumple la política.
RF-10	El sistema deberá autenticar a los clientes por correo y contraseña, y a los administradores por nombre de usuario y contraseña, y abrir una sesión.	M	R11	Credenciales correctas abren sesión; incorrectas se rechazan con mensaje genérico.

ID	Requerimiento	Prior.	Origen	Criterio de aceptación
RF-11	El sistema deberá permitir a un cliente autenticado reservar un paquete indicando la cantidad de personas, y registrar la fecha de emisión y el total.	M	R12 – R16	Cantidad ≥ 1 ; se rechaza sobre el cupo disponible o con salida pasada; total = precio $\times$ personas, inmodificable.
RF-12	El sistema deberá permitir al cliente consultar el historial de sus propias reservas.	M	R11	Un cliente solo ve sus reservas; nunca las de otro.
RF-13	El sistema deberá restringir las operaciones según el rol: solo administradores gestionan el catálogo y solo clientes autenticados reservan.	M	R11, S1	Un cliente que intenta gestionar destinos, o una sesión inexistente o expirada, reciben un rechazo de autorización.
RF-14	El sistema deberá validar todo dato ingresado (tipo, formato, rango y longitud) antes de procesarlo.	M	Alcance	Entradas inválidas o con código SQL se rechazan sin alterar la base de datos.
RF-15	El sistema no deberá mostrar el RUT ni el teléfono completo en listados ni en mensajes de error.	M	R17	Los datos aparecen enmascarados y los mensajes de error no los incluyen.
RF-16	El sistema deberá permitir al cliente cancelar una reserva propia de un paquete aún no iniciado, liberando el cupo.	S	S4	La reserva pasa a CANCELADA y el cupo disponible aumenta; su total no se modifica.
RF-17	El sistema deberá permitir al administrador listar todos los paquetes, incluidos los vencidos.	S	S3, entrevista a M. Bórquez	El administrador ve el estado Vigente / Vencido de cada paquete.
RF-18	El sistema deberá impedir que un mismo cliente mantenga dos reservas activas en el mismo paquete.	M	S6, 19 duplicadas	El segundo intento se rechaza; tras cancelar, puede volver a reservar.
RF-19	El sistema deberá permitir al administrador agregar salidas adicionales a un paquete, cada una con su fecha de salida, fecha de regreso y cupo.	S	S9	La salida nueva tiene su propio cupo; las salidas anteriores conservan sus fechas, cupo y reservas.
RF-20	El sistema deberá permitir al cliente restablecer su contraseña indicando su correo y su RUT.	C	S11	Con datos correctos se actualiza la contraseña; tras 5 intentos fallidos la cuenta se bloquea 15 minutos y el rechazo usa siempre el mismo mensaje.
RF-21	El sistema deberá permitir al administrador listar todas las reservas registradas.	S	S5	El administrador ve todas las reservas; un cliente que lo intenta recibe un rechazo de autorización.
RF-22	El sistema deberá permitir a un administrador crear otros administradores y cambiar sus propias credenciales.	S	S1	Solo una sesión de administrador puede hacerlo; la contraseña nueva debe cumplir la política.

2.4 Requerimientos no funcionales

Los requerimientos no funcionales (RNF) describen cómo debe comportarse el sistema en seguridad, integridad, rendimiento, usabilidad, mantenibilidad y portabilidad. Se redactan con la misma estructura que los funcionales, con su categoría, prioridad MoSCoW y un criterio de verificación. El método y la

evidencia con que se comprobó cada uno se presentan en la Tabla 10. Los requerimientos no funcionales se listan en la Tabla 4.

Tabla 4. Requerimientos no funcionales (RNF).

ID	Categoría	Requerimiento	Prior.	Criterio de verificación
RNF-01	Seguridad	Las contraseñas deberán almacenarse solo como hash Argon2id con sal; nunca tal como las escribe el cliente (R10).	M	La base de datos no contiene la contraseña y el hash comienza con \$argon2id\$.
RNF-02	Confidencialidad	El RUT y el teléfono deberán almacenarse cifrados en reposo con cifrado autenticado (Fernet) (R17).	M	El campo almacenado no contiene el dato en claro y solo se descifra con la clave.
RNF-03	Seguridad	Toda consulta SQL deberá ser parametrizada para evitar inyección SQL.	M	Las cargas de ataque (por ejemplo, una comilla seguida de OR 1=1) no alteran ni exponen datos.
RNF-04	Seguridad de acceso	El acceso deberá bloquear la cuenta durante 15 minutos tras 5 intentos fallidos, usar mensajes genéricos y expirar la sesión tras 30 minutos de inactividad.	M	La cuenta se bloquea y se libera por tiempo; el mensaje es el mismo para correo inexistente y clave incorrecta.
RNF-05	Integridad	Las reglas críticas deberán reforzarse en la base de datos (CHECK, UNIQUE, FOREIGN KEY) y las reservas deberán ejecutarse en una transacción para que el cupo sea consistente.	M	El esquema contiene las restricciones y la reserva verifica el cupo y guarda de forma atómica.
RNF-06	Rendimiento	Las operaciones de catálogo y de reserva deberán responder en menos de 1 segundo con el volumen de una temporada (aproximadamente 214 reservas).	S	Medición con la base de demostración (tiempos en la tabla de verificación de RNF).
RNF-07	Usabilidad	La interfaz de consola deberá usar menús numerados y la interfaz web formularios, ambas con mensajes claros en español y sin mostrar trazas técnicas al usuario.	S	Los errores inesperados se registran en el log y al usuario se le informa con un mensaje genérico.
RNF-08	Mantenibilidad	El código deberá organizarse en capas, con programación orientada a objetos y pruebas automatizadas de las reglas de negocio.	M	49 pruebas automatizadas aprobadas y módulos de responsabilidad única.
RNF-09	Portabilidad	El sistema deberá ejecutarse en Windows, Linux o macOS con Python 3.10 o superior y sin instalar un motor de base de datos.	M	Base de datos SQLite en un archivo y dependencias declaradas en requirements.txt.
RNF-10	Trazabilidad	El sistema deberá registrar los errores técnicos en un archivo de registro sin incluir datos sensibles, y guardar en la base de datos los intentos fallidos de acceso de cada cuenta.	S	El filtro del registro enmascara RUT, teléfono y contraseñas; el archivo app.log no se sube al repositorio.
RNF-11	Exactitud	Los montos deberán manejarse como enteros (CLP) y el margen con aritmética Decimal, evitando errores de coma flotante.	M	Precios y totales exactos en las pruebas de cálculo.

ID	Categoría	Requerimiento	Prior.	Criterio de verificación
RNF-12	Seguridad web	La sesión web deberá mantenerse en una cookie no accesible desde scripts y restringida al mismo sitio, y toda operación que modifique datos deberá exigir una cabecera de verificación.	M	Sin una cookie de sesión válida la API responde 401; sin la cabecera responde 403.

2.5 Clasificación, priorización y verificación

Clasificación. Los requerimientos se organizaron en tres ejes: gestión de catálogo (RF-01 a RF-08, RF-17 y RF-19), reservas y clientes (RF-09 a RF-12, RF-16, RF-18, RF-20 y RF-21) y seguridad transversal (RF-13 a RF-15, RF-22, RNF-01 a RNF-04, RNF-10 y RNF-12). Los requerimientos de calidad restantes (RNF-05 a RNF-09 y RNF-11) se asocian al conjunto de la solución.

Criterios de priorización. Se aplicó MoSCoW con tres criterios: (1) valor, si elimina una de las fallas medidas en la temporada; (2) riesgo, si compromete datos sensibles o dinero; y (3) dependencia técnica, si otros requerimientos no pueden construirse sin él (por ejemplo, la autenticación RF-10 precede a la reserva RF-11). La priorización resultante se muestra en la Tabla 5.

Tabla 5. Priorización MoSCoW de los requerimientos.

Prioridad	Requerimientos	Justificación
Debe (M)	RF-01 a RF-15 y RF-18; RNF-01 a RNF-05, RNF-08, RNF-09, RNF-11 y RNF-12	Resuelven directamente las fallas medidas en la temporada (sobrecupo, fechas vencidas, precios distintos, duplicados) y protegen los datos sensibles; sin ellos el sistema no cumple el alcance.
Debería (S)	RF-16, RF-17, RF-19, RF-21 y RF-22; RNF-06, RNF-07 y RNF-10	Mejoran la operación y cubren los vacíos del caso (desistimiento, paquetes vencidos, salidas múltiples), pero la primera versión puede operar sin ellos..
Podría (C)	RF-20	La recuperación de contraseña aporta comodidad al cliente, pero no resuelve ninguna de las fallas medidas en la temporada.
No se hará (W)	Pasarela de pago, facturación electrónica, aplicación móvil, integración con operadores externos, envío de correos, informes de gestión y contabilidad	Los socios los dejaron fuera del alcance de la primera versión (sección 1.3). Se declaran para evitar que se reclamen después.

Verificación frente a la problemática (Indicador 4.1.1.I.4). Cada evidencia de la sección 2.1 tiene al menos un requerimiento que elimina su causa raíz, y cada regla de negocio R1 a R17 está cubierta por un requerimiento con criterio de aceptación (Tabla 6):

Tabla 6. Cobertura de las reglas de negocio R1 a R17 por requerimiento.

Regla	Cubierta por	Regla	Cubierta por	Regla	Cubierta por
R1	RF-01	R7	RF-07	R13	RF-11
R2	RF-01, RF-02	R8	RF-03, RF-04	R14	RF-08, RF-11
R3	RF-05	R9	RF-09	R15	RF-08, RF-11
R4	RF-05	R10	RF-09, RNF-01	R16	RF-11
R5	RF-05	R11	RF-10, RF-12, RF-13	R17	RF-15, RNF-02
R6	RF-06	R12	RF-11	—	—

Lo declarado fuera de alcance (pago automático, facturación, correos, informes) no genera requerimientos. Las expectativas de los tres socios entrevistados quedan cubiertas: Paulina (cálculo automático del precio y baja lógica de destinos), Matías (control de cupo y de fechas vencidas) e Ignacio (datos protegidos, conteo e historial consultable).

2.6 Supuestos por vacíos del caso

La Tabla 7 reúne los supuestos adoptados para cerrar los vacíos del caso, con el fundamento de cada uno.

Tabla 7. Supuestos adoptados ante los vacíos del caso.

ID	Vacío detectado	Supuesto adoptado	Fundamento técnico
S1	¿Quién puede modificar el catálogo?	Existe el rol ADMINISTRADOR. El primer administrador se crea al iniciar la aplicación por primera vez, con credenciales definidas en variables de entorno; los siguientes solo puede crearlos otro administrador. Los clientes no pueden gestionar destinos ni paquetes..	Principio de mínimo privilegio; evita que un cliente altere precios. Un administrador no se autorregistra para impedir escalamiento de privilegios.
S2	¿Cuándo “se publica” un paquete?	Crear un paquete equivale a publicarlo: el precio se calcula y se congela en ese momento.	Simplifica la versión 1 sin perder la regla R7; un estado borrador puede agregarse después.
S3	¿Qué ocurre con un paquete cuando termina su temporada?	El paquete no se borra ni cambia: queda vencido al pasar su fecha de salida. No admite reservas y el cliente no lo ve; el administrador sí.	Conserva el historial de reservas y evita el error de vender paquetes vencidos (R15). El estado se deriva de la fecha, no se almacena, por lo que no puede quedar desactualizado.
S4	¿Qué pasa cuando un cliente desiste?	El cliente puede cancelar su reserva antes de la salida: pasa a CANCELADA, libera el cupo y conserva su registro. Reembolsos quedan fuera del sistema.	No se borran filas (auditoría); el cupo solo cuenta reservas ACTIVAS. Coherente con “sin pasarela de pago”.
S5	¿Cómo se considera el pago?	El pago por transferencia se verifica manualmente fuera del sistema; la reserva nace ACTIVA.	La verificación automática está fuera de alcance.
S6	¿Puede un cliente reservar dos veces el mismo paquete?	Solo una reserva activa por cliente en cada salida de un paquete; para cambiar la cantidad de personas se cancela y se reserva de nuevo.	Evita duplicados como los 19 detectados; se refuerza con un trigger en la base de datos.
S7	¿Moneda y formato de montos?	Pesos chilenos como enteros; margen con Decimal.	Evita errores de redondeo de coma flotante en precios.
S8	¿Límites del margen?	Margen entre 0 % y 500 %.	La regla solo exige “no negativo”; el tope protege de errores de digitación (como el cero de más reportado).
S9	¿La “temporada” es una entidad?	No es una entidad propia. Un paquete puede tener varias salidas, cada una con sus fechas y su cupo; la temporada se deduce de esas fechas.	El caso muestra paquetes que se repiten (“Todo el año”, “Enero y febrero”). Las salidas permiten repetir un paquete sin duplicarlo ni recalculer su precio.
S10	¿Tipo de interfaz y despliegue?	Dos interfaces sobre la misma base SQLite local: una de consola y una web, servida desde el propio equipo.	Tres socios y unas 214 reservas por temporada no justifican un servidor externo; ambas interfaces usan la misma capa de servicios, por lo que las reglas se aplican igual en las dos.
S11	¿Cómo recupera su contraseña un cliente?	Indicando su correo y su RUT, sin envío de correos. Los intentos fallidos cuentan para el mismo bloqueo del inicio de sesión.	El envío de correos está fuera del alcance. El RUT no es un dato secreto, por lo que se limita a 5 intentos y se declara como riesgo residual en la sección 6.

3. Modelado de la solución (criterio 4.1.2)

El modelado representa la solución desde tres perspectivas complementarias: la interacción entre usuarios y sistema (casos de uso), los procesos de negocio (BPMN) y la estructura orientada a objetos y de datos (diagramas de clases y modelo entidad-relación). Los identificadores RF, CU, BPMN y R permiten rastrear cada elemento hasta su requerimiento (sección 3.6).

3.1 Diagrama de casos de uso

Participan dos actores, los mismos roles del caso: Cliente y Administrador. La autenticación (CU-02) está incluida en todas las operaciones protegidas, y el cálculo del precio (CU-09) forma parte de la creación del paquete. El registro (CU-01) y la recuperación de contraseña (CU-12) son los únicos casos que no exigen una sesión iniciada. La Figura 1 muestra los actores y sus casos de uso.

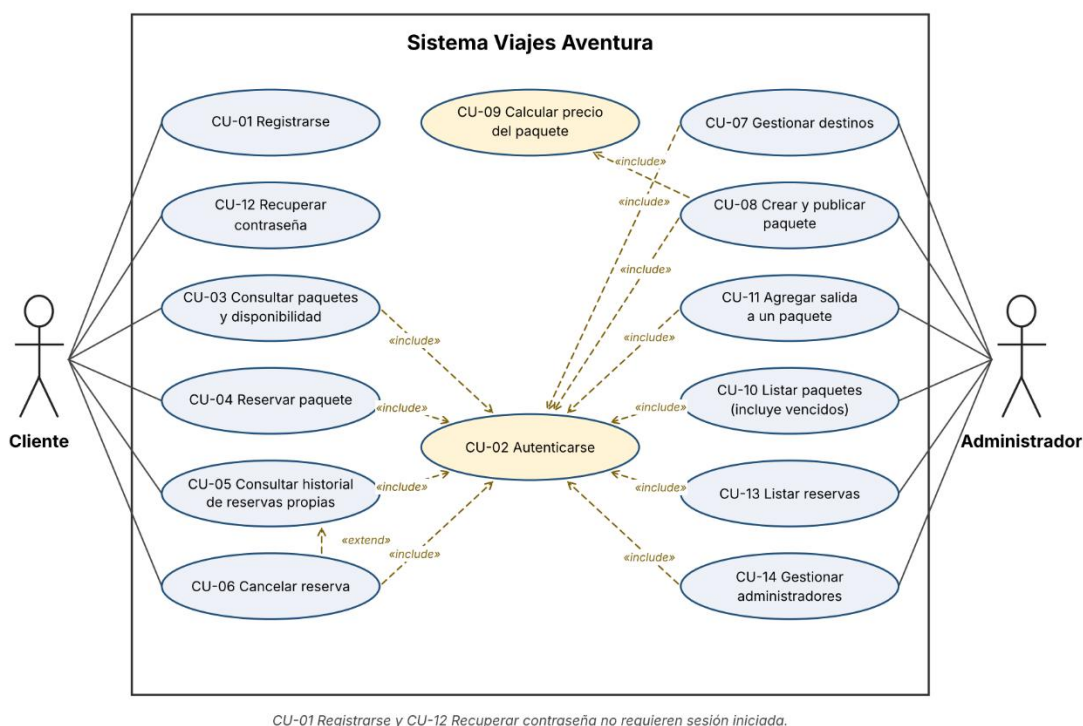


Figura 1. Diagrama de casos de uso de Viajes Aventura (UML).

La Tabla 8 especifica cada caso de uso: actor, precondition, flujo principal y requerimientos asociados.

Tabla 8. Especificación de los casos de uso.

ID	Caso de uso	Actor	Precondición	Flujo principal y resultado	Requerimientos
CU-01	Registrarse	Cliente	Ninguna	Ingresa nombre, RUT, correo, teléfono y contraseña; el sistema valida, cifra datos sensibles, aplica hash a la clave y crea el cliente.	RF-09, RF-14, RNF-01, RNF-02
CU-02	Autenticarse	Cliente / Administrador	Usuario registrado	Ingresa su correo (cliente) o su nombre de usuario (administrador) y la contraseña; el sistema verifica bloqueo y hash, y abre una sesión con rol. Falla con mensaje genérico.	RF-10, RF-13, RNF- 04

ID	Caso de uso	Actor	Precondición	Flujo principal y resultado	Requerimientos
CU-03	Consultar paquetes y disponibilidad	Cliente	Sesión activa	Lista los paquetes vigentes con destinos, precio por persona y cupo disponible.	RF-08
CU-04	Reservar paquete	Cliente	Sesión activa; paquete vigente con cupo	Elige paquete y cantidad; el sistema valida reglas R14 – R16, calcula el total y guarda la reserva de forma atómica.	RF-11, RF-18, RNF- 05
CU-05	Consultar historial de reservas	Cliente	Sesión activa	Muestra solo las reservas del cliente autenticado, con RUT y teléfono ocultos.	RF-12, RF-15
CU-06	Cancelar reserva	Cliente	Reserva propia activa de un paquete no iniciado	Cambia la reserva a CANCELADA y libera el cupo.	RF-16
CU-07	Gestionar destinos	Administrador	Sesión de administrador	Registra, modifica, lista y elimina o desactiva destinos según R8.	RF-01 a RF-04, RF- 13
CU-08	Crear y publicar paquete	Administrador	Sesión de administrador; ≥ 2 destinos disponibles	Elige 2 a 5 destinos, fechas, cupo y margen; el sistema valida y publica con precio congelado.	RF-05, RF-07
CU-09	Calcular precio del paquete	Administrador (incluido en CU-08)	Destinos elegidos	Precio = suma de costos base + margen (Decimal).	RF-06, RNF-11
CU-10	Listar paquetes (incluye vencidos)	Administrador	Sesión de administrador	Muestra todos los paquetes con su estado Vigente / Vencido y cupos.	RF-17
CU-11	Agregar salida a un paquete	Administrador	Sesión de administrador; paquete existente	Elige un paquete e indica fecha de salida, fecha de regreso y cupo; el sistema valida y agrega la salida sin alterar las anteriores.	RF-19
CU-12	Recuperar contraseña	Cliente	Cuenta registrada	Ingresa correo, RUT y contraseña nueva; el sistema verifica y actualiza la contraseña. Tras 5 intentos fallidos la cuenta se bloquea 15 minutos.	RF-20
CU-13	Listar reservas	Administrador	Sesión de administrador	Muestra todas las reservas con paquete, cantidad de personas, total, estado y fechas, sin datos sensibles del cliente.	RF-21
CU-14	Gestionar administradores	Administrador	Sesión de administrador	Crea otro administrador o cambia sus propias credenciales; la contraseña debe cumplir la política.	RF-22

3.2 Modelos de procesos de negocio (BPMN)

Se modelaron cinco procesos con carriles para cada participante (usuario y sistema). Los eventos de inicio son círculos verdes; los de fin, círculos gruesos (con una X cuando el proceso termina en rechazo); las compuertas exclusivas (rombos) representan las decisiones que implementan las reglas de negocio. Los cinco procesos se presentan en las Figuras 2 a 6.

BPMN 1 · Registro de cliente

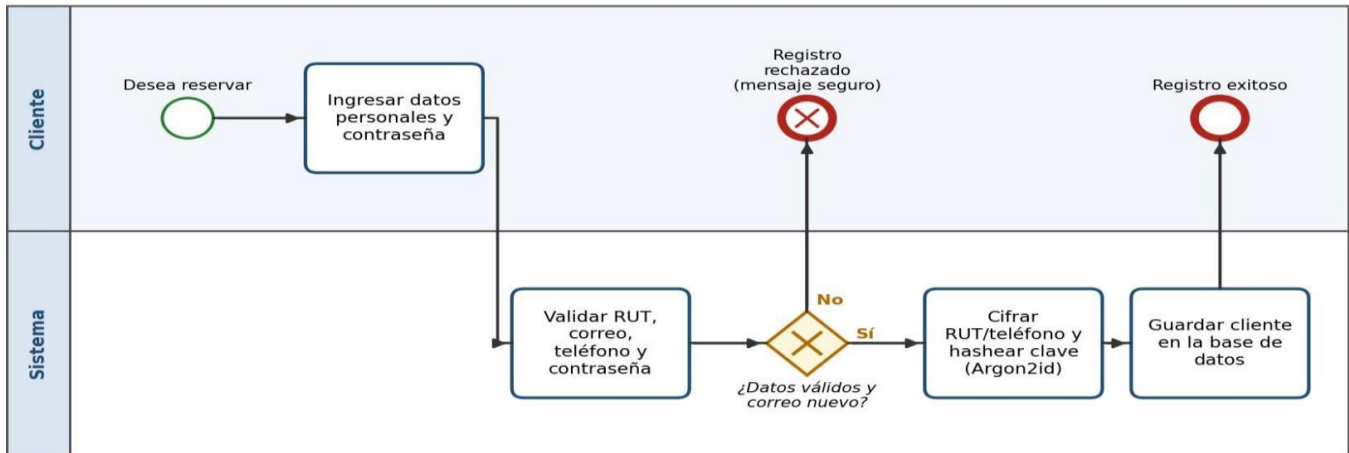


Figura 2. BPMN 1 · Registro de cliente (RF-09, R9, R10, R17).

BPMN 2 · Autenticación de usuario

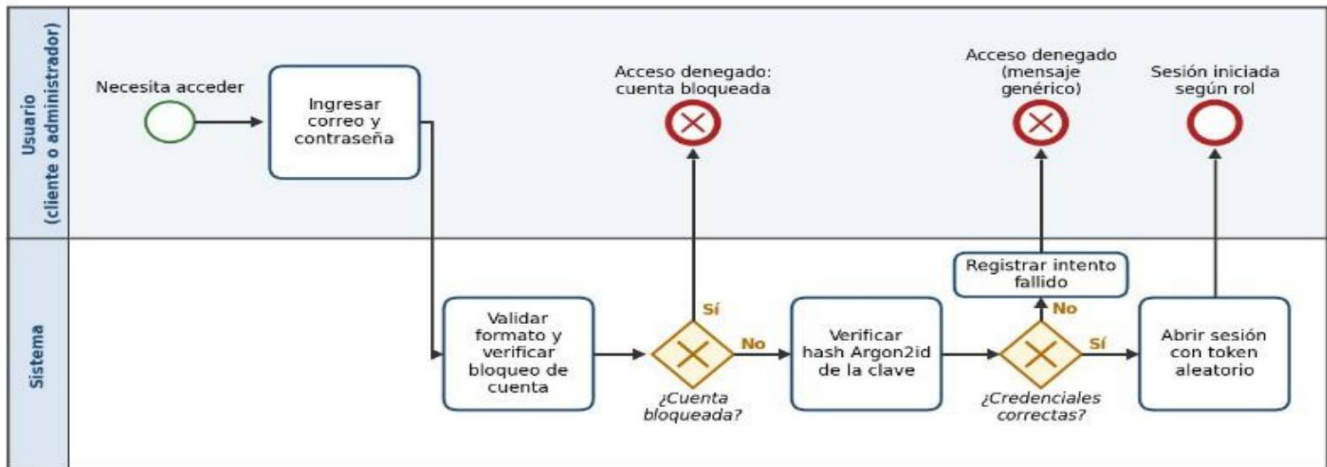


Figura 3. BPMN 2 · Autenticación (RF-10, RNF-04, R11).

BPMN 3 · Gestión de paquetes: crear y publicar

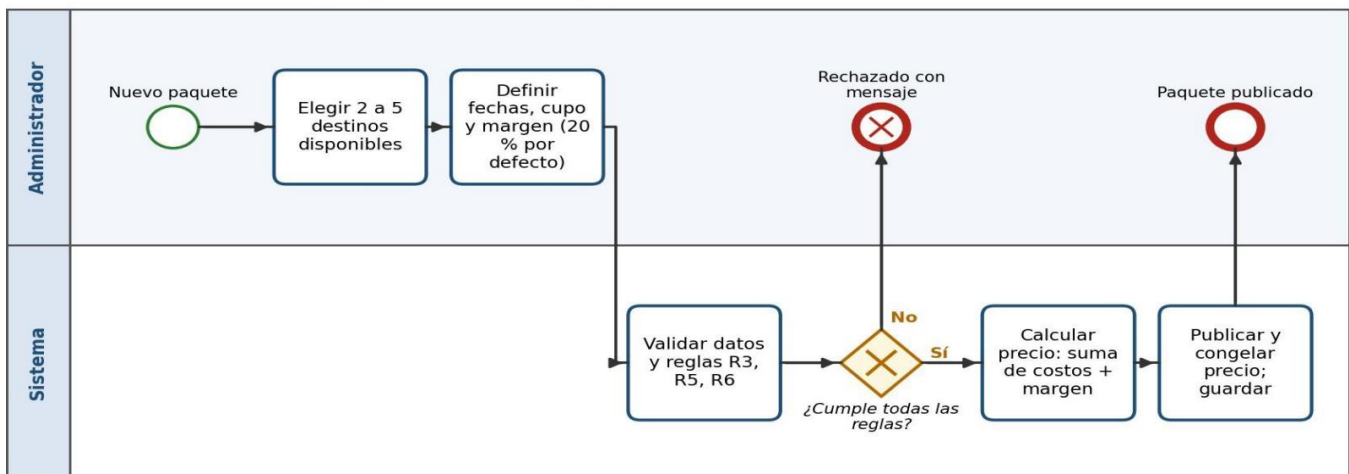


Figura 4. BPMN 3 · Gestión de paquetes: crear y publicar (RF-05, RF-06, RF-07; R3, R5, R6, R7).

BPMN 4 · Gestión de destinos: eliminar o desactivar

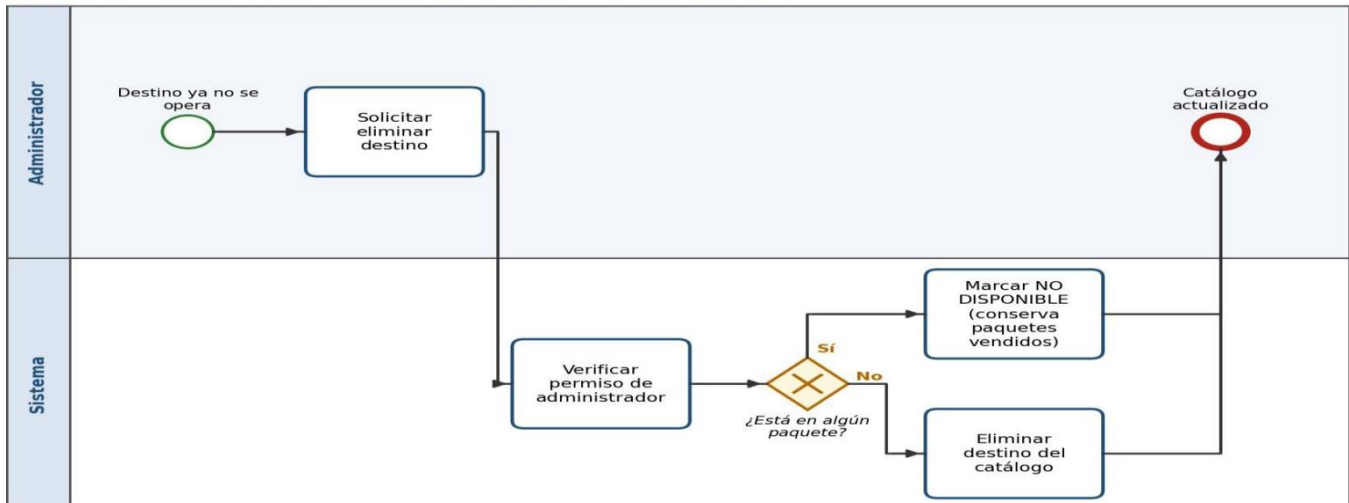


Figura 5. BPMN 4 · Gestión de destinos: eliminar o desactivar (RF-03, R8).

BPMN 5 · Reserva de un paquete turístico

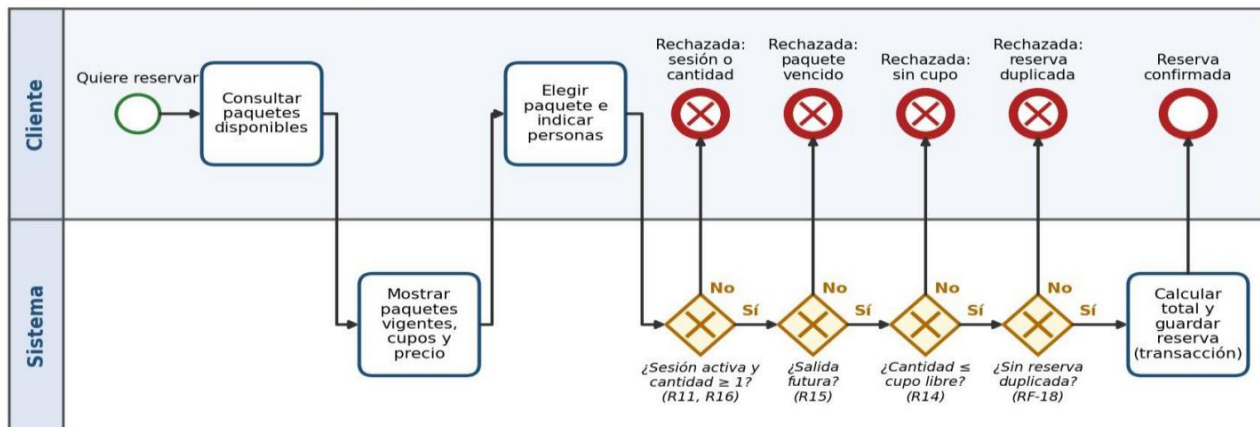


Figura 6. BPMN 5 · Reserva de un paquete (RF-08, RF-11, RF-18; R12 a R16).

Lectura de las decisiones. En el proceso de reserva (Figura 6), cuatro compuertas rechazan la solicitud en orden: (1) sesión activa y cantidad mínima (R11, R16); (2) fecha de salida contra la fecha del día (R15); (3) cantidad solicitada contra el cupo disponible que el sistema calcula (R14); y (4) que el cliente no tenga ya una reserva activa en el mismo paquete (RF-18). Las compuertas 2 y 3 se separan porque, como señala la pista del caso, comparan datos distintos: la fecha del día y un cupo que el sistema debe llevar. Solo si las cuatro respuestas son afirmativas se calcula el total y se guarda la reserva en una transacción (R13). En la baja de destinos (Figura 5), la compuerta decide entre eliminar o desactivar (R8), lo que evita el problema de Paulina de “filas sin nada” en paquetes ya vendidos.

3.3 Diagramas de clases UML

3.3.1 Clases de dominio

El dominio aplica abstracción (Usuario es una clase abstracta), herencia (Cliente y Administrador heredan de ella), polimorfismo (cada subclase responde de forma distinta a puede_reservar()) y encapsulamiento (atributos privados expuestos mediante propiedades de solo lectura, con validación al

construir el objeto). Las multiplicidades reflejan las reglas: un paquete combina de 2 a 5 destinos (R3) y un destino puede estar en varios paquetes (R4); un cliente realiza muchas reservas y cada reserva corresponde a un único paquete (R12). Un paquete tiene una o más salidas, y cada reserva se asocia a una de ellas. El diagrama se presenta en la Figura 7.

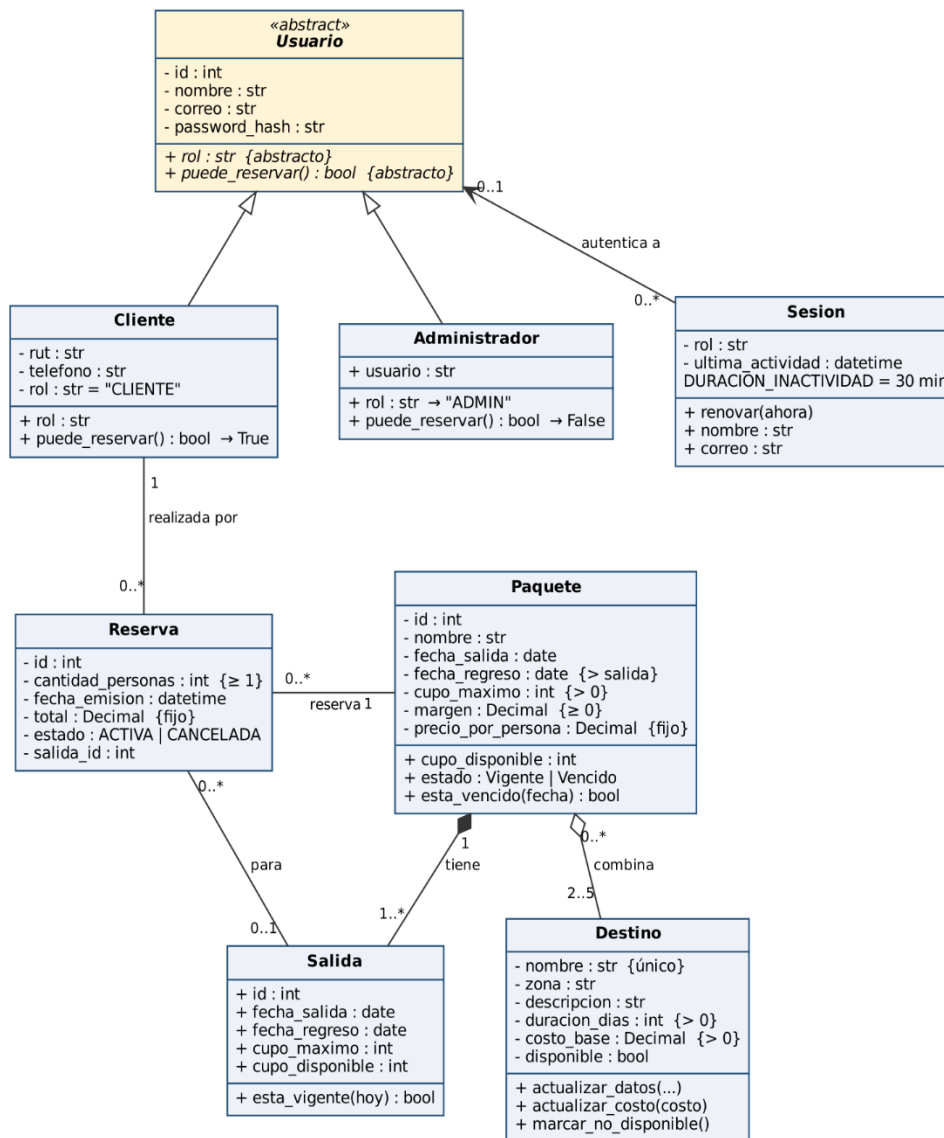


Figura 7. Diagrama de clases de dominio.

Lectura del diagrama. Usuario concentra los datos comunes (identificador, nombre, correo y hash de la contraseña) y declara como abstractos rol y puede_reservar(), por lo que no puede instanciarse. La relación entre Paquete y Destino es una agregación: el destino existe por sí mismo y lo comparten varios paquetes. La relación entre Paquete y Salida es una composición: una salida no existe sin su paquete. Se optó por una jerarquía de clases y no por un único Usuario con un atributo de rol, porque así la regla «solo un cliente reserva» queda en el propio objeto y no en comparaciones repartidas por el código.

3.3.2 Clases de servicios, acceso a datos y seguridad

La lógica de negocio vive en cuatro servicios que colaboran con cinco repositorios (acceso a datos) y con los componentes de seguridad. Las dos interfaces, consola y web, construyen los servicios y delegan

en ellos todas las reglas de negocio; por eso la interfaz web pudo agregarse sin modificarlas. El diagrama se presenta en la Figura 8.

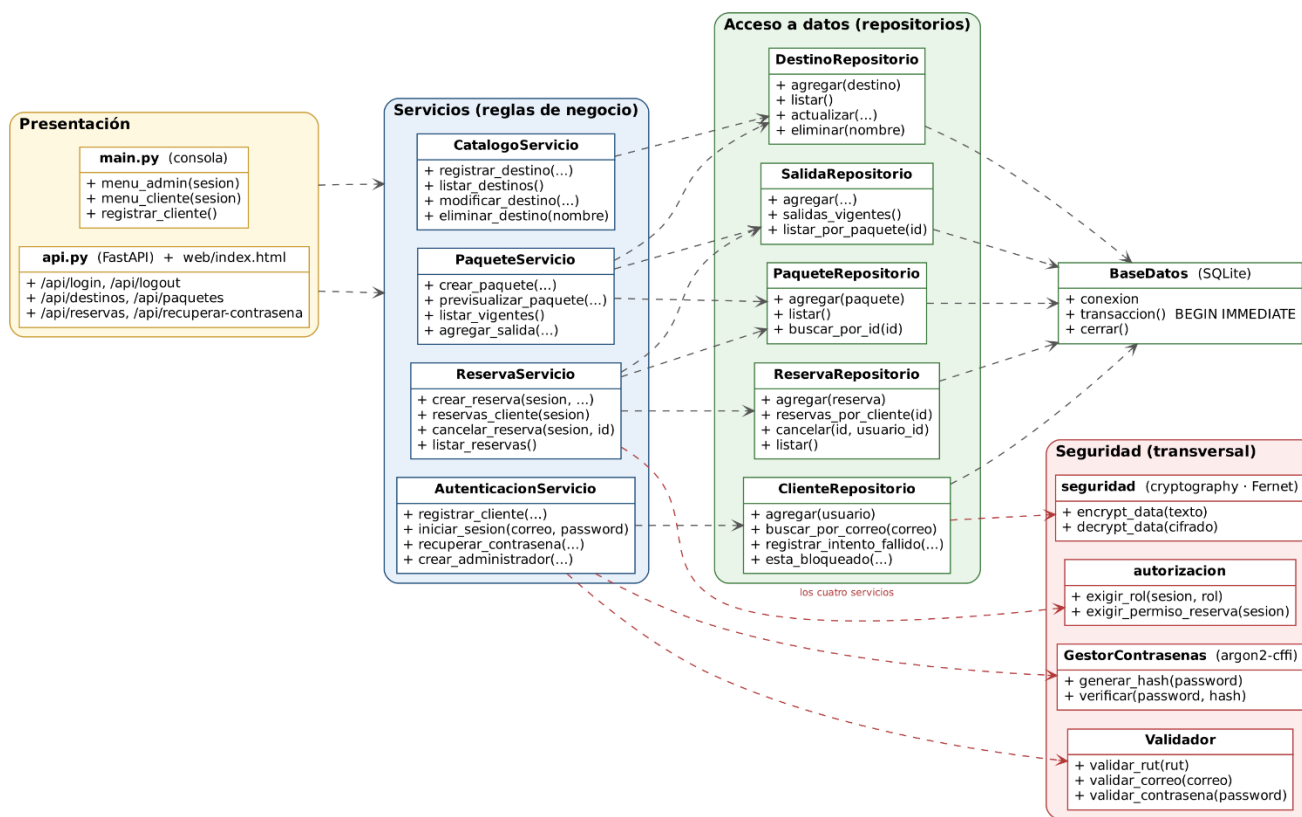
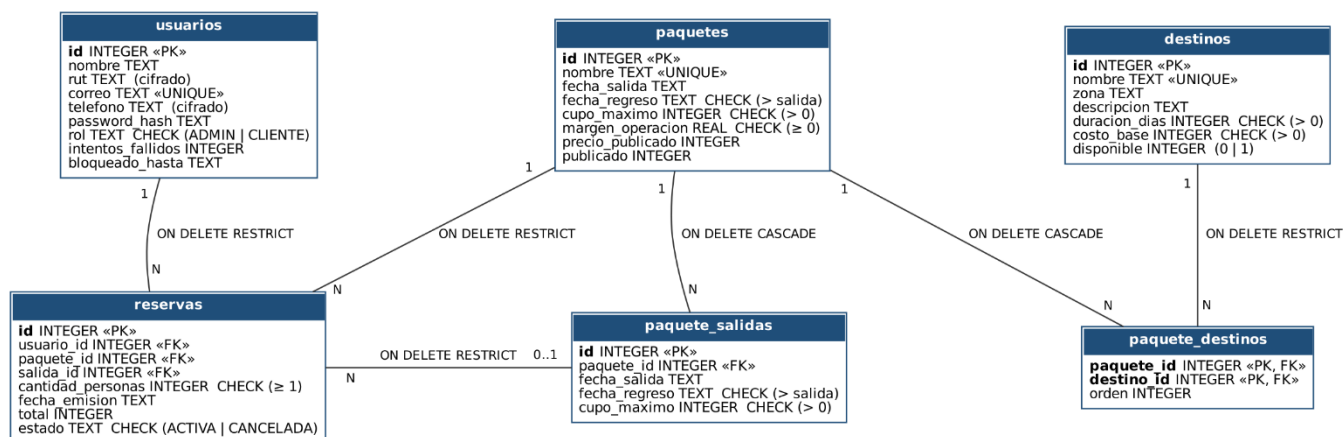


Figura 8. Diagrama de clases de servicios, repositorios y seguridad.

Lectura del diagrama. Cada servicio recibe sus repositorios al construirse, lo que permite probarlos con una base de datos temporal. Todos los repositorios comparten una misma BaseDatos, que entrega la conexión y las transacciones, y son el único lugar del sistema con consultas SQL. Se descartó usar un ORM: con seis tablas y reglas simples, las consultas SQL parametrizadas son más fáciles de leer y de defender.

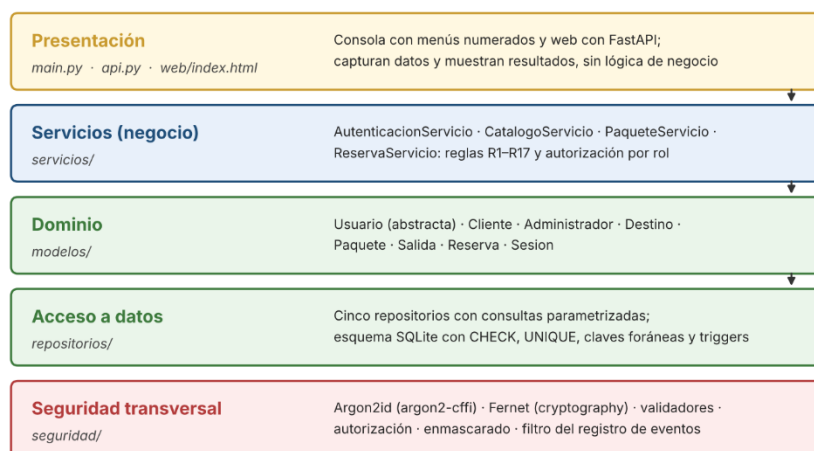
3.4 Modelo de datos

La base de datos SQLite refuerza las reglas críticas con restricciones propias (CHECK, UNIQUE, claves foráneas y triggers), de modo que aunque una capa superior fallara, los datos inválidos no podrían persistirse. La tabla paquete_destinos resuelve la relación de muchos a muchos entre paquetes y destinos; su clave foránea hacia destinos usa ON DELETE RESTRICT, que impide borrar un destino usado por un paquete (R8). El precio se fija en la columna precio_publicado de la tabla paquetes, por lo que un cambio posterior en el costo de un destino no lo altera (R7). La tabla paquete_salidas permite que un mismo paquete tenga varias fechas, cada una con su cupo. El RUT y el teléfono se guardan cifrados, y los montos como enteros en pesos chilenos. El modelo entidad-relación se presenta en la Figura 9.



Lectura del diagrama. Son seis tablas. Los triggers impiden superar el cupo, duplicar la reserva activa de un mismo cliente y dejar un paquete con menos de 2 o más de 5 destinos. Se prefirió reforzar esas reglas en la base de datos, y no controlarlas solo en el código, porque así se cumplen aunque dos operaciones ocurran al mismo tiempo.

La solución se organiza en cinco capas. Cada capa usa solo las que tiene debajo, y la seguridad es transversal: la emplean los servicios y los repositorios. Las capas se muestran en la Figura 10.



Lectura del diagrama. La presentación tiene dos entradas, consola y web, que comparten los mismos servicios. Los servicios aplican las reglas R1 a R17 y la autorización; el dominio contiene las entidades y sus validaciones; los repositorios concentran las consultas SQL. Se eligió separar en capas, y no reunir todo en un solo archivo, porque permitió agregar la interfaz web sin tocar las reglas y probar cada regla sin levantar la interfaz.

3.6 Trazabilidad entre requerimientos y modelos (Indicador 4.1.2.I.8)

La matriz vincula cada requerimiento con los modelos que lo representan, el código que lo implementa y la prueba automatizada que lo verifica. No existe requerimiento sin modelo, ni modelo sin requerimiento.

Tabla 9. Matriz de trazabilidad entre requerimientos, modelos, código y pruebas.

Req.	Caso de uso	BPMN	Clases / métodos principales	Tablas	Prueba automatizada (carpeta tests/)
RF-01 (R1, R2)	CU-07	BPMN 4	Destino (validaciones) CatalogoServicio.registrar_destino DestinoRepositorio.agregar	destinos	test_destinos.py: test_registro_valida_datos_y_rechaza_nombre_duplicado_sin_case
RF-02 (R7)	CU-07	—	CatalogoServicio.modificar_destino Destino.actualizar_datos DestinoRepositorio.actualizar	destinos	test_destinos.py: test_modificar_destino_no_cambia_precio_de_paquete_publicado
RF-03 (R8)	CU-07	BPMN 4	CatalogoServicio.eliminar_destino DestinoRepositorio.eliminar Destino.marcar_no_disponible	destinos, paquete_destinos	test_destinos.py: test_elimina_destino_sin_paquetes_y_desactiva_destino_asociado
RF-04 (R8)	CU-07	BPMN 4	CatalogoServicio.listar_destinos PaqueteServicio.crear_paquete (rechaza un destino no disponible)	destinos	test_paquetes.py: test_rechaza_selecciones_invalidas_y_margen_negativo
RF-05 (R3, R4, R5)	CU-08	BPMN 3	Paquete (validaciones) PaqueteServicio.crear_paquete PaqueteRepositorio.agregar	paquetes, paquete_destinos, paquete_salidas	test_paquetes.py: test_rechaza_selecciones_invalidas_y_margen_negativo test_persistencia.py: test_paquete_exige_entre_dos_y_cinco_destinos
RF-06 (R6)	CU-09	BPMN 3	Paquete (cálculo del precio) PaqueteServicio.previsualizar_paquete aplicar_margen_clp (seguridad/montos.py)	paquetes	test_paquetes.py: test_calcula_redondea_y_conserva_precio_publicado test_seguridad.py: test_montos_clp_usando_decimal_y_redondeo_half_up
RF-07 (R7)	CU-08	BPMN 3	Paquete.precio_por_persona columna precio_publicado PaqueteRepositorio.agregar	paquetes	test_paquetes.py: test_calcula_redondea_y_conserva_precio_publicado test_destinos.py: test_modificar_destino_no_cambia_precio_de_paquete_publicado
RF-08 (R14, R15)	CU-03	BPMN 5	PaqueteServicio.listar_vigentes PaqueteRepositorio.paquetes_vigentes Paquete.cupo_disponible	paquetes, paquete_salidas, reservas	test_paquetes.py: test_consultas_cliente_y_admin_exponen_cupo_y_estado
RF-09 (R9, R10)	CU-01	BPMN 1	AutenticacionServicio.registrar_cliente Validador hash_password y encrypt_data (seguridad/seguridad.py) ClienteRepositorio.agregar	usuarios	test_persistencia.py: test_registro_exige_rut_mod11_y_correo_unico test_api_registro.py: test_registra_y_no_devuelve_datos_credenciales
RF-10 (R11)	CU-02	BPMN 2	AutenticacionServicio.iniciar_sesion Sesion verify_password	usuarios	test_autenticacion.py: test_credenciales_invalidas_siempre_tienen_mensaje_generico test_persiste_y_recupera_rol_admin

Req.	Caso de uso	BPMN	Clases / métodos principales	Tablas	Prueba automatizada (carpeta tests/)
RF-11 (R12 – R16)	CU-04	BPMN 5	ReservaServicio.crear_reserva Reserva (cálculo del total) ReservaRepositorio.agregar	reservas, paquete_salidas	test_reservas.py: test_reserva_por_id_guarda_fecha_actual_y_total_publicado test_rechaza_cantidad_invalida_paquete_inexistente_y_fecha_pasada test_cupo_se_calcula_con_reservas_activas_en_transaccion
RF-12 (R11)	CU-05	—	ReservaServicio.reservas_cliente ReservaRepositorio.reservas_por_cliente	reservas	test_reservas.py: test_historial_aislado_por_usuario_y_cancelacion_propia
RF-13 (R11)	CU-02	BPMN 2	exigir_rol y exigir_permiso_reserva (seguridad/autorizacion.py) Usuario.puede_reservar	usuarios	test_destinos.py: test_cliente_no_puede_gestionar_destinos test_paquetes.py: test_rol_no_pueden_invocar_operaciones_de_paquetes_ajenas test_rubrica.py: testuml_usuario_cliente_administrador_y_permisos
RF-14	Todos	Todos	Validador (seguridad/validadores.py) validaciones de Destino, Paquete y Reserva	CHECK de todas las tablas	test_seguridad.py: test_validadores_de_rut_mod11_y_password_con_caracter_especial test_api_registro.py: test_responde_422_para_rut_o_password_invalidos test_persistencia.py: test_constraints_sql_relacion_y_cupo_transaccional
RF-15 (R17)	CU-05	BPMN 1	Enmascarador, mask_rut y mask_phone (seguridad/enmascarado.py) FiltroDatosSensibles	usuarios	test_presentacion.py: test_cliente_se_muestra_con_rut_y_telefono_enmascarados test_seguridad.py: test_mascaras_de_rut_y_telefono test_api_registro.py: test_enmascara_rut_y_telefono_en_errores_http
RF-16	CU-06	—	ReservaServicio.cancelar_reserva ReservaRepositorio.cancelar	reservas	test_reservas.py: test_historial_aislado_por_usuario_y_cancelacion_propia test_rechaza_cancelacion_el_dia_de_inicio
RF-17	CU-10	—	PaqueteServicio.listar_paquetes Paquete.estado	paquetes	test_paquetes.py: test_consultas_cliente_y_admin_exponen_cupo_y_estado
RF-18	CU-04	BPMN 5	Triggers reserva_unica_salida_insert y reserva_unica_paquete_insert ReservaRepositorio.agregar	reservas	test_reservas.py: test_impide_duplicado_activo_y_permite_reservar_tras_cancelacion
RF-19	CU-11	—	PaqueteServicio.agregar_salida Salida SalidaRepositorio.agregar	paquete_salidas	test_paquetes.py: test_agregar_salida_conserva_la_salida_anterior
RF-20	CU-12	—	AutenticacionServicio.recuperar_contraseña ClienteRepositorio.actualizar_password_hash	usuarios	test_rubrica.py: test_recuperacion_se_bloquea_tras_cinco_rut_incorrectos
RF-21	CU-13	—	ReservaServicio.listar_reservas ReservaRepositorio.listar	reservas	test_persistencia.py: test_cliente_paquete_y_reserva_sobreviven_reapertura

Req.	Caso de uso	BPMN	Clases / métodos principales	Tablas	Prueba automatizada (carpeta tests/)
RF-22	CU-14	—	AutenticacionServicio.crear_administrador AutenticacionServicio.cambiar_credenciales_administrador	usuarios	test_autenticacion.py: test_admin_puede_crear_otro_y_cambiar_sus_credenciales

Validación de coherencia

Se comprobó que: (1) los 14 casos de uso cubren los 22 requerimientos funcionales; (2) cada decisión (compuerta) de los BPMN corresponde a una regla del caso y a un método del código; (3) todas las clases de los diagramas de clases existen en el código con los mismos nombres y relaciones; (4) las reglas críticas tienen además una restricción o un trigger en la base de datos; y (5) cada requerimiento funcional tiene al menos una prueba automatizada aprobada (Tabla 9) y cada requerimiento no funcional tiene un método de verificación, por prueba o por inspección (Tabla 10). Se declaran tres excepciones: la regla R4 no tiene una prueba dedicada, y los requerimientos RNF-03 y RNF-12 se verifican por inspección y no por prueba automatizada.

Tabla 10. Verificación de los requerimientos no funcionales.

RNF	Método de verificación	Evidencia
RNF-01	Prueba automatizada	test_seguridad.py: test_hash_argon2id_verifica_sin_guardar_clave test_persistencia.py: test_login_migra_hash_pbkdf2_a_argon2id
RNF-02	Prueba automatizada	test_persistencia.py: test_rut_y_telefono_se_guardan_cifrados test_seguridad.py: test_cifrado_fernet_hace_round_trip
RNF-03	Inspección del código de la carpeta repositorios/	Todas las consultas pasan los datos como parámetros (?); ninguna arma el texto SQL con datos del usuario. No tiene una prueba automatizada dedicada.
RNF-04	Prueba automatizada	test_autenticacion.py: test_bloquea_al_quinto_fallo_y_permite_login_a_los_15_minutos, test_sesion_expira_tras_30_minutos_inactiva test_rubrica.py: test_correo_inexistente_y_cuenta_bloqueada_comparten_mensaje
RNF-05	Inspección del esquema (CHECK, UNIQUE, claves foráneas y triggers) y prueba automatizada	test_persistencia.py: test_constraints_sql_relacion_y_cupo_transaccional test_pytest_suite.py: test_cupo_no_se_sobrecarga_con_reservas_concurrentes
RNF-06	Medición con time. perf_counter sobre una base temporal con 20 paquetes y 214 reservas (script medir_rendimiento.py)	Medición del 5 de octubre de 2026: listado de paquetes vigentes, 2,3 ms; reserva de un paquete, 5,7 ms. Ambas bajo el límite de 1 segundo.
RNF-07	Prueba automatizada e inspección de la consola y de la página web	test_presentacion.py: test_menu_principal_original_muestra_opciones_numeradas, test_error_tecnico_muestra_mensaje_generico_y_registra_traza
RNF-08	Prueba automatizada e inspección de la estructura de carpetas	Las 49 pruebas de la carpeta tests/ (py -m pytest -q). Carpetas modelos/, servicios/, repositorios/ y seguridad/.
RNF-09	Inspección de las dependencias y del motor de base de datos	requirements.txt; SQLite del módulo estándar sqlite3.
RNF-10	Prueba automatizada	test_seguridad.py: test_filtro_enmascara_datos_sensibles

RNF	Método de verificación	Evidencia
RNF-11	Prueba automatizada	test_seguridad.py: test_montos_clp_usando_decimal_y_redondeo_half_up test_paquetes.py: test_calcula_redondea_y_conserva_precio_publicado
RNF-12	Inspección de api.py y comprobación manual de las respuestas	La cookie de sesión se crea con HttpOnly y SameSite=strict. Una petición sin cookie recibe 401 (Figura 15). La cabecera X-Requested-With se exige en api.py, que responde 403 si falta. No tiene una prueba automatizada dedicada.

4. Planificación con metodologías ágiles (criterio 4.1.3)

4.1 Marco de trabajo y roles

Se trabajó con un marco ágil basado en Scrum, adaptado a un equipo de tres personas y a un plazo real de dos semanas. Se mantuvieron sus elementos centrales: un backlog priorizado, iteraciones cortas que terminan en un incremento ejecutable y la revisión de cada incremento con la suite de pruebas. No se realizaron las ceremonias formales de Scrum con duración fija: la coordinación fue continua y por mensajería, lo que se declara aquí como una adaptación. Los roles y el aporte de cada integrante se detallan en la Tabla 11.

Tabla 11. Roles y responsabilidades del equipo.

Rol	Responsable	Responsabilidades y aporte realizado
Líder de Proyecto y Scrum Master	Benjamín Comanato	Coordina al equipo y mantiene la comunicación con el docente. Aportó uno de los dos avances iniciales de código e informe. En el código: la jerarquía Usuario, Cliente y Administrador, el hash señuelo, la política de contraseñas y ajustes de la interfaz web.
Product Owner y desarrollador principal	Sergio Ruiz	Prioriza el backlog. Aportó el otro avance inicial. Desarrolló el núcleo del sistema (esquema SQLite, servicios, seguridad, consola y suite de pruebas) y la interfaz web, y redactó la versión base del informe. Registró en changelog.md cada pedido hecho a las herramientas de IA.
Análisis, modelado y calidad	Bernardo Fuentes	Se incorporó el 30 de septiembre. Revisó la coherencia entre el informe y el código, amplió el informe, rehízo los diagramas de casos de uso, clases, datos y arquitectura, corrigió el código en ocho commits (polimorfismo, recuperación de contraseña, datos personales en el repositorio y trazabilidad) y ejecutó las pruebas manuales de la interfaz web.

Criterio de asignación. El reparto siguió el punto de partida del proyecto. Sergio Ruiz y Benjamín Comanato llegaron con avances propios de código e informe, que se integraron el 29 de septiembre; por eso el desarrollo quedó en manos de quienes ya conocían esa base. Bernardo Fuentes se incorporó después y asumió la revisión, que exigía leer el sistema completo con mirada externa. La concentración del código en una sola cuenta (cerca de tres cuartos de los commits) es una debilidad del proceso, que se declara en las conclusiones; se compensó con la revisión cruzada final, para que cada integrante pueda explicar el sistema completo en su defensa.

Coordinación y acuerdos

- Coordinación continua por mensajería (WhatsApp): avances, bloqueos y reparto de tareas.
- Revisión de cada incremento: tras cada cambio se ejecuta la suite de pruebas y el resultado queda anotado en changelog.md (por ejemplo, «44/44 tests pasan sin regresiones»).
- Registro del uso de IA: cada pedido, la respuesta y las pruebas ejecutadas se anotan en changelog.md.
- Definición de Hecho: pruebas automatizadas aprobadas, cambio subido al repositorio, regla del caso trazada y ningún dato sensible en el repositorio. Los dos últimos puntos no se cumplieron siempre durante el desarrollo y se regularizaron en la iteración 3.

4.2 Product Backlog priorizado

Las historias de usuario derivan de los requerimientos (columna Req.). Los puntos expresan esfuerzo relativo y la prioridad sigue MoSCoW. La columna Estado funciona como tablero al cierre del proyecto, el 5 de octubre de 2026: las 25 historias quedaron hechas, cuatro de ellas con observaciones. El backlog suma 119 puntos. El detalle se presenta en la Tabla 12.

Tabla 12. Product Backlog priorizado y estado al cierre.

ID	Historia de usuario	Req.	Prior.	Pts	Iter.	Estado
HU-01	Como analista, quiero requerimientos y supuestos documentados para construir lo que el negocio necesita.	RF-01 a RF-22	M	5	1	Hecha
HU-02	Como equipo, quiero modelos UML y BPMN para compartir una visión común y trazable.	Todos	M	8	1	Hecha
HU-03	Como equipo, quiero un esquema de base de datos con restricciones para proteger la integridad.	RNF-05	M	5	1	Hecha
HU-04	Como administrador, quiero registrar y listar destinos para mantener el catálogo.	RF-01, RF-04	M	5	1	Hecha
HU-05	Como administrador, quiero modificar un destino sin alterar los paquetes ya publicados.	RF-02, RF-07	M	3	1	Hecha
HU-06	Como administrador, quiero eliminar o desactivar un destino según esté o no en paquetes.	RF-03	M	5	1	Hecha
HU-07	Como administrador, quiero crear un paquete con 2 a 5 destinos, fechas, cupo y margen.	RF-05	M	8	1	Hecha
HU-08	Como administrador, quiero que el sistema calcule el precio y lo fije al publicar.	RF-06, RF-07	M	5	1	Hecha
HU-09	Como cliente, quiero registrarme con datos protegidos para reservar con seguridad.	RF-09, RNF-01, RNF-02	M	8	1	Hecha
HU-10	Como usuario, quiero iniciar sesión con bloqueo ante intentos fallidos y permisos por rol.	RF-10, RF-13, RNF-04	M	8	1	Hecha
HU-11	Como cliente, quiero consultar paquetes vigentes con cupo y precio.	RF-08	M	3	1	Hecha
HU-12	Como cliente, quiero reservar un paquete sin superar el cupo ni reservar fechas vencidas.	RF-11, RF-18	M	8	1	Hecha
HU-13	Como cliente, quiero consultar mi historial de reservas con mis datos sensibles ocultos.	RF-12, RF-15	M	3	1	Hecha
HU-14	Como cliente, quiero cancelar una reserva para liberar el cupo si desisto.	RF-16	S	3	1	Hecha

ID	Historia de usuario	Req.	Prior.	Pts	Iter.	Estado
HU-15	Como administrador, quiero ver todos los paquetes, incluidos los vencidos.	RF-17	S	2	1	Hecha
HU-16a	Como equipo, quiero validar las entradas del catálogo y probar las reglas de destinos y paquetes (R1 a R8).	RF-14, RNF-08	M	3	1	Hecha
HU-16b	Como equipo, quiero validar las entradas de registro y reserva y probar las reglas de autenticación, cupo y fechas (R9 a R17).	RF-14, RNF-08	M	3	1	Hecha
HU-21	Como administrador, quiero ver todas las reservas para dar seguimiento a las ventas.	RF-21	S	2	1	Hecha
HU-22	Como administrador, quiero crear otros administradores y cambiar mis credenciales.	RF-22	S	3	1	Hecha
HU-23	Como usuario, quiero una interfaz web para operar el sistema desde el navegador.	RNF-07, RNF-12	S	8	2	Hecha, con observación
HU-19	Como administrador, quiero agregar salidas a un paquete para ofrecerlo en otras fechas sin cambiar su precio.	RF-19	S	3	2	Hecha
HU-20	Como cliente, quiero recuperar mi contraseña con mi correo y mi RUT.	RF-20	C	3	2	Hecha, con observación
HU-16c	Como equipo, quiero completar la cobertura y ejecutar toda la suite con evidencia de aprobación.	RF-14, RNF-08	M	2	3	Hecha, con observación
HU-17	Como equipo, quiero evaluar la seguridad y revisar críticamente el código generado con IA.	RNF-03, RNF-10	M	5	3	Hecha, con observación
HU-18	Como equipo, quiero el informe técnico y la preparación de la defensa individual.	—	M	8	3	Hecha

Observaciones al cierre: la interfaz web (HU-23) funciona solo en el mismo equipo y no ofrece exactamente las mismas operaciones que la consola; la recuperación de contraseña (HU-20) queda con un riesgo medio declarado; la suite (HU-16c) no tiene una prueba dedicada para la regla R4; y en la evaluación de seguridad (HU-17) los requerimientos RNF-03 y RNF-12 se verificaron por inspección.

4.3 Sprint Backlog por iteración

Entre el 22 y el 28 de septiembre hubo trabajo previo: dos integrantes avanzaron por separado en código y en borradores del informe, fuera del repositorio. Desde el 29 de septiembre el trabajo se organizó en tres iteraciones. La Tabla 13 muestra lo que ocurrió, con fechas y cifras tomadas del historial del repositorio.

Tabla 13. Sprint Backlog por iteración.

Iteración	Objetivo	Historias y tareas realizadas	Incremento entregado	Pts
1 · 29 sep – 2 oct Integración y núcleo	Unir los dos avances y dejar el sistema completo en consola	Repositorio compartido y reglas para el agente de IA (gemini.md); primera versión de requerimientos y modelos (HU-01, HU-02); esquema SQLite con restricciones (HU-03); destinos, paquetes y precio (HU-04 a HU-08); registro, autenticación y roles (HU-09, HU-10, HU-22); consulta, reserva, historial y cancelación (HU-11 a HU-14, HU-21); listado para el administrador (HU-15); pruebas de cada parte (HU-16a, HU-16b).	Aplicación de consola completa con persistencia; 44 pruebas aprobadas; primera versión del informe	90
2 · 3 – 4 oct Interfaz web y ampliaciones	Agregar la web sin duplicar las reglas	Segunda versión del informe; interfaz web con FastAPI (HU-23); recuperación de contraseña (HU-20); salidas adicionales por paquete (HU-19); jerarquía Usuario, hash señuelo y política de contraseñas.	Aplicación web local sobre los mismos servicios; 48 pruebas aprobadas	14
3 · 4 – 5 oct Revisión, seguridad y entrega	Hacer coincidir el informe con el código y entregar	Revisión de coherencia sección por sección; ocho commits de corrección (limpieza del repositorio, polimorfismo, límite en la recuperación de contraseña, trazabilidad y script de rendimiento); evaluación de seguridad y registro del uso de IA (HU-17); suite completa con evidencia (HU-16c); pruebas manuales de la web; informe final y preparación de la defensa (HU-18); carga en AAI antes de las 23:00 h.	49 pruebas aprobadas; informe final; código en el repositorio	15

Los puntos suman 119 (90 + 14 + 15). La carga no es pareja: la iteración 1 concentra la mayor parte porque consolidó en el repositorio el trabajo previo de dos integrantes. Es la principal lección de proceso y se recoge en las conclusiones.

4.4 Tiempos, entregables y riesgos

La Tabla 14 presenta el cronograma real del proyecto, con el entregable y la evidencia de cada hito, y la Tabla 15 resume los riesgos previstos, su mitigación y lo que ocurrió con cada uno.

Tabla 14. Cronograma, hitos y entregables.

Fecha (2026)	Hito	Entregable	Responsable	Evidencia
22 – 28 sep	Trabajo previo	Avances individuales de código e informe	Sergio Ruiz y Benjamín Comanato, por separado	Fuera del repositorio
29 sep	Integración	Repositorio compartido con la base común	Sergio Ruiz	Primer commit del repositorio
1 oct	Primera versión del informe	Informe en Word	Equipo	Archivo subido al repositorio
2 oct	Fin de la iteración 1	Consola completa con persistencia	Sergio Ruiz	15 commits; 44 pruebas aprobadas
3 oct	Segunda versión del informe	Informe en PDF	Sergio Ruiz	Commit del 3 de octubre
4 oct	Fin de la iteración 2	Interfaz web y ampliaciones	Sergio Ruiz y Benjamín Comanato	Commits del 4 de octubre; 48 pruebas aprobadas
5 oct	Entrega final	Informe técnico, código fuente y aplicación funcional	Líder de Proyecto	49 pruebas aprobadas (Figura 11); carga en AAI antes de las 23:00 h
Posterior	Defensa	Defensa argumentativa individual	Cada integrante	Explica su aporte, las decisiones, el uso de IA, la seguridad y la persistencia

Tabla 15. Riesgos del proyecto, mitigación y resultado.

Riesgo del proyecto	Prob.	Impacto	Mitigación	Qué ocurrió
Integración tardía del trabajo de los integrantes	Media	Alto	Repositorio compartido desde el 29 de septiembre	Ocurrió en parte: el código quedó concentrado en un integrante. Se compensó con la revisión cruzada final
Código de IA no comprendido o con fallos de seguridad	Media	Alto	Pruebas automatizadas tras cada cambio y registro de decisiones (sección 7)	Ocurrió: de ocho propuestas registradas, cuatro se modificaron y dos se descartaron (Tabla 26)
Informe que no coincide con el código	Media	Alto	Revisión de coherencia contra el repositorio	Ocurrió: la primera versión describía clases y pruebas inexistentes. Se corrigió en la iteración 3
Ambigüedad del caso	Alta	Medio	Supuestos explícitos S1 – S11	Controlado: cada vacío del caso tiene un supuesto declarado
Falla de carga en AAI de último minuto	Baja	Alto	Cargar con horas de margen y comprobar que el PDF abre	Se verifica al momento de la entrega

5. Implementación en Python con persistencia (criterio 4.1.4)

5.1 Tecnologías y organización del código

La Tabla 16 resume las tecnologías utilizadas y la razón de cada elección.

Tabla 16. Tecnologías utilizadas y su justificación.

Componente	Elección	Justificación
Lenguaje	Python 3.14 (compatible con 3.10 o superior)	Lenguaje del curso; clases, propiedades y excepciones propias (excepciones.py).
Persistencia	SQLite con el módulo estándar sqlite3	Sin servidor; adecuada a 3 socios y unas 214 reservas por temporada; soporta transacciones, claves foráneas, restricciones CHECK y triggers.
Hash de contraseñas	argon2-cffi (PyPI)	Argon2id, algoritmo recomendado por OWASP; la sal y los parámetros van incluidos en el hash.
Cifrado de datos	cryptography · Fernet (PyPI)	Cifrado simétrico autenticado: confidencialidad más detección de alteración.
Interfaz de consola	main.py con menús numerados	No requiere navegador ni servidor; fue la primera interfaz y sirve para la operación básica.
Interfaz web	FastAPI y uvicorn (api.py) con una página HTML y JavaScript sin bibliotecas externas (web/index.html)	Expone por HTTP los mismos servicios de la consola; FastAPI valida el formato de cada petición antes de procesarla.
Pruebas	pytest (con httpx para probar la API)	49 pruebas ejecutables con un comando: <code>py -m pytest -q</code> .

5.2 Programación orientada a objetos aplicada

La Tabla 17 muestra dónde se aplica cada principio de la programación orientada a objetos, con un ejemplo tomado del código.

Tabla 17. Aplicación de los principios de programación orientada a objetos.

Principio	Dónde se aplica	Ejemplo concreto
Abstracción	Usuario es una clase abstracta (ABC), en <code>modelos/usuario.py</code>	Declara <code>rol</code> y <code>puede_reservar()</code> como abstractos: no se puede crear un Usuario sin definir de qué tipo es.
Encapsulamiento	Atributos privados con propiedades de solo lectura en Usuario, Destino, Paquete y Reserva	Destino valida sus datos al crearse y rechaza un costo base menor o igual a cero (R2); un objeto inválido no puede existir.
Herencia	Cliente y Administrador heredan de Usuario	Ambos reutilizan <code>id</code> , <code>nombre</code> , <code>correo</code> y <code>password_hash</code> ; Cliente agrega <code>RUT</code> y <code>teléfono</code> .
Polimorfismo	<code>exigir_permiso_reserva</code> (<code>seguridad/autorizacion.py</code>) llama a <code>usuario.puede_reservar()</code>	Cliente responde verdadero y Administrador responde falso; el servicio de reservas no pregunta de qué tipo es el usuario.
Agregación y composición	Paquete agrupa Destinos (agregación) y tiene Salidas (composición)	Un destino existe aunque no pertenezca a ningún paquete; una salida se elimina junto con su paquete (ON DELETE CASCADE).
Responsabilidad única	Cada servicio recibe sus repositorios al construirse	<code>ReservaServicio(reservas, paquetes)</code> no crea su propia conexión; por eso las pruebas pueden entregarle una base de datos en memoria.

5.3 Persistencia de datos

El esquema (sección 3.4) se crea automáticamente al abrir la base de datos (clase `BaseDatos`, en `repositorios/db_setup.py`). Las reglas se aplican en dos niveles: en el dominio, que entrega mensajes claros al usuario, y en la base de datos, con restricciones y triggers como segunda barrera. Las operaciones que dependen de un dato compartido, como el cupo, se ejecutan dentro de una transacción con `BEGIN IMMEDIATE`, que bloquea a otros escritores y evita la sobreventa que sufrió Matías. La

persistencia se comprobó cerrando y reabriendo la base de datos en la prueba `test_cliente_paquete_y_reserva_sobreviven_reapertura`.

5.4 Funcionalidades CRUD

La Tabla 18 indica qué operaciones existen para cada entidad y quién puede ejecutarlas.

Tabla 18. Matriz CRUD por entidad.

Entidad	Crear	Leer	Actualizar	Eliminar	Actor
Destino	registrar_destino	listar_destinos	modificar_destino	eliminar_destino: física si está libre; pasa a «no disponible» si está en un paquete (R8)	Administrador
Paquete	crear_paquete (publica y fija el precio)	listar_paquetes (administrador) listar_vigentes (cliente) previsualizar_paquete	agregar_salida: nuevas fechas y cupo; el precio publicado no cambia (R7)	No aplica: se conserva como historial (S3)	Administrador; el cliente solo lee los vigentes
Reserva	crear_reserva	reservas_cliente (cliente) listar_reservas (administrador)	cancelar_reserva: cambia el estado; el total no cambia (R13)	No se elimina: se cancela (S4)	Cliente; el administrador solo lista
Usuario	registrar_cliente crear_administrador	iniciar_sesion (verificación)	recuperar_contrasena cambiar_credenciales_administrador	No incluido en el alcance	Cliente y Administrador

Las dos interfaces comparten los mismos servicios, pero no ofrecen exactamente las mismas operaciones. Modificar y eliminar destinos, previsualizar el precio, agregar salidas y recuperar la contraseña están disponibles solo en la interfaz web; crear administradores y cambiar sus credenciales, solo en la consola. El resto de las operaciones está en ambas.

5.5 Pruebas y verificación

Se escribieron 49 pruebas automatizadas, que se ejecutan con `pytest` mediante el comando `py -m pytest -q`. Cada una se asocia a una regla del caso o a un control de seguridad (Tabla 9). Usan bases de datos en memoria o temporales y una hora simulada para comprobar bloqueos y expiraciones sin esperar, por lo que son repetibles. Resultado de la ejecución final: 49 pruebas aprobadas y ningún fallo (Figura 11).

```
PS C:\Users\nardo\Viajeaventura> py -m pytest -q
.....uuuuuuuu.....uuuu.....uuuu. [100%]
===== warnings summary =====
.. \AppData\Local\Programs\Python\Python314\Lib\site-packages\fastapi\testclient.py:1
C:\Users\nardo\AppData\Local\Programs\Python\Python314\Lib\site-packages\fastapi\testclient.py:1: StarletteDeprecation
Warning: Using 'httpx' with 'starlette.testclient' is deprecated; install 'httpx2' instead.
  from starlette.testclient import TestClient as TestClient # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
49 passed, 1 warning, 14 subtests passed in 4.81s
PS C:\Users\nardo\Viajeaventura>
```

Figura 11. Ejecución de la suite: 49 pruebas aprobadas.

La Tabla 19 agrupa las pruebas por archivo e indica qué verifica cada grupo.

Tabla 19. Pruebas automatizadas por grupo.

Grupo	Archivo	Qué verifican
Destinos (4)	test_destinos.py	Datos válidos y nombre único (R1, R2); un cliente no gestiona destinos; modificar un destino no cambia el precio publicado (R7); eliminar o desactivar (R8).
Paquetes (5)	test_paquetes.py	Cálculo y precio fijo (R6, R7); permisos por rol; selecciones inválidas y margen negativo (R3, R5); cupo y estado en las consultas; agregar una salida.
Reservas (6)	test_reservas.py	Fecha y total (R12, R13); cantidad inválida y fecha pasada (R15, R16); cupo (R14); reserva duplicada; historial aislado por cliente (R11); cancelación el día de inicio.
Autenticación (5)	test_autenticacion.py	Mensaje genérico; bloqueo tras 5 intentos durante 15 minutos; expiración de la sesión a los 30 minutos; rol de administrador; crear administradores y cambiar credenciales.
Seguridad (6)	test_seguridad.py	Hash Argon2id; cifrado Fernet; montos con Decimal; filtro del registro de eventos; enmascarado; validadores de RUT y contraseña.
Persistencia (8)	test_persistencia.py	RUT y teléfono cifrados en la base; RUT módulo 11 y correo único; migración de datos de versiones anteriores (3 pruebas); los datos sobreviven al reabrir la base; restricciones SQL y cupo en transacción; entre 2 y 5 destinos.
API web (4)	test_api_registro.py	El registro no devuelve credenciales; 409 para correo duplicado; 422 para datos inválidos; los errores no muestran RUT ni teléfono.
Consola y presentación (5)	test_presentacion.py test_configuracion_consola.py	Datos enmascarados al mostrar un cliente; menú numerado; error técnico con mensaje genérico; margen leído como porcentaje; error de configuración de la clave de cifrado.
Rúbrica (4)	test_rubrica.py	Jerarquía Usuario, Cliente y Administrador; límite superior de la contraseña; mismo mensaje para cuenta bloqueada; bloqueo en la recuperación de contraseña.
Concurrencia (2)	test_pytest_suite.py	Aislamiento entre pruebas; el cupo no se supera con reservas simultáneas.

Además, el 5 de octubre de 2026 se verificó manualmente el flujo completo en la interfaz web, con los seis destinos de demostración. Se creó un paquete con Valle del Elqui y Salar de Surire, cuyo precio calculado fue de \$516.000 por persona (Figura 12). Un cliente registrado reservó para 2 personas: el total fue de \$1.032.000 y el cupo disponible bajó de 4 a 2 (Figura 13). Un segundo intento de reserva sobre la misma salida fue rechazado (Figura 14). Al cancelar, la reserva quedó en estado CANCELADA, conservó su total y el cupo volvió a 4. El registro del servidor muestra el código de cada petición, incluidos el 401 sin sesión y el 409 de la reserva duplicada (Figura 15). En la consola se comprobó que un destino con duración 0 se rechaza con un mensaje claro y sin interrumpir el programa.

Destinos
Paquetes
Reservas

Paquetes

#	Nombre y destinos	Precio p/p	Salida	Regreso	Cupo máx	Usados	Disponibles	Estado
1	Valle del Elqui Valle del Elqui, Salar de Surire	\$516.000	15/11/2026	20/11/2026	4	0	4	Vigente

Crear paquete

Nombre del paquete

Destinos (separados por coma, entre 2 y 5)

Fecha de salida

Fecha de regreso

Cupo máximo

Figura 12. Paquete publicado con su precio de \$516.000 por persona.

Viajes Aventura
Ana Soto
Salir

Paquetes disponibles

Nombre y destinos	Precio p/p	Salida	Regreso	Cupos
Valle del Elqui Valle del Elqui, Salar de Surire	\$516.000	15/11/2026	20/11/2026	2

Mis reservas

#	Paquete	Salida	Personas	Total	Estado
1	Valle del Elqui	15/11/2026	2	\$1.032.000	ACTIVA

Figura 13. Reserva confirmada y cupo actualizado.

Viajes Aventura
Ana Soto
Salir

Paquetes disponibles

Nombre y destinos	Precio p/p	Salida	Regreso	Cupos	
Valle del Elqui Valle del Elqui, Salar de Surire	\$516.000	15/11/2026	20/11/2026	2	Reservar

Personas: 1
Total: \$516.000
Confirmar
Cancelar
Ya tienes una reserva activa para esta salida.

Mis reservas

#	Paquete	Salida	Personas	Total	Estado	
1	Valle del Elqui	15/11/2026	2	\$1.032.000	ACTIVA	Cancelar

Figura 14. Rechazo de una reserva duplicada.

```

PS C:\Users\nardo\Viajeaventura> py -m uvicorn api:app --reload
INFO: Will watch for changes in these directories: ['C:\\Users\\nardo\\Viajeaventura']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [18944] using StatReload
INFO: Started server process [6856]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:63203 - "GET / HTTP/1.1" 200 OK
INFO: 127.0.0.1:63203 - "GET /api/yo HTTP/1.1" 401 Unauthorized
INFO: 127.0.0.1:63203 - "GET /favicon.ico HTTP/1.1" 404 Not Found
INFO: 127.0.0.1:61796 - "POST /api/login HTTP/1.1" 200 OK
INFO: 127.0.0.1:61796 - "GET /api/destinos HTTP/1.1" 200 OK
INFO: 127.0.0.1:59451 - "GET /api/paquetes HTTP/1.1" 200 OK
INFO: 127.0.0.1:58878 - "POST /api/paquetes/previsualizar HTTP/1.1" 422 Unprocessable Content
INFO: 127.0.0.1:62137 - "POST /api/paquetes/previsualizar HTTP/1.1" 200 OK
INFO: 127.0.0.1:56276 - "POST /api/paquetes HTTP/1.1" 201 Created
INFO: 127.0.0.1:56276 - "GET /api/paquetes HTTP/1.1" 200 OK
INFO: 127.0.0.1:64215 - "POST /api/logout HTTP/1.1" 200 OK
INFO: 127.0.0.1:54571 - "POST /clientes/registro HTTP/1.1" 422 Unprocessable Content
INFO: 127.0.0.1:56112 - "POST /clientes/registro HTTP/1.1" 422 Unprocessable Content
INFO: 127.0.0.1:55199 - "POST /clientes/registro HTTP/1.1" 201 Created
INFO: 127.0.0.1:50625 - "POST /api/login HTTP/1.1" 401 Unauthorized
INFO: 127.0.0.1:55553 - "POST /api/login HTTP/1.1" 200 OK
INFO: 127.0.0.1:50704 - "GET /api/paquetes HTTP/1.1" 200 OK
INFO: 127.0.0.1:55553 - "GET /api/reservas HTTP/1.1" 200 OK
INFO: 127.0.0.1:56232 - "POST /api/reservas HTTP/1.1" 201 Created
INFO: 127.0.0.1:57607 - "GET /api/paquetes HTTP/1.1" 200 OK
INFO: 127.0.0.1:56232 - "GET /api/reservas HTTP/1.1" 200 OK
INFO: 127.0.0.1:53046 - "POST /api/reservas HTTP/1.1" 409 Conflict
INFO: 127.0.0.1:52827 - "POST /api/reservas/1/cancelar HTTP/1.1" 200 OK
INFO: 127.0.0.1:55254 - "GET /api/paquetes HTTP/1.1" 200 OK
INFO: 127.0.0.1:52827 - "GET /api/reservas HTTP/1.1" 200 OK
INFO: Shutting down
INFO: Waiting for application shutdown.
INFO: Application shutdown complete.
INFO: Finished server process [6856]
INFO: Stopping reloader process [18944]
  
```

Figura 15. Registro del servidor durante el recorrido de prueba.

5.6 Limitaciones conocidas

- La interfaz web se ejecuta en el mismo equipo (http://127.0.0.1:8000) y sin HTTPS; publicarla en internet exigiría un servidor con certificado.
- Las sesiones web se guardan en la memoria del servidor: si este se reinicia, los usuarios deben iniciar sesión otra vez.
- Las dos interfaces no ofrecen las mismas operaciones (ver el párrafo que sigue a la Tabla 18).
- SQLite atiende a un volumen bajo y a pocos usuarios simultáneos; como las consultas SQL están solo en los repositorios, migrar a PostgreSQL cambiaría únicamente esa capa.

- La regla R4 y los requerimientos RNF-03 y RNF-12 no tienen una prueba automatizada dedicada; se verifican por inspección (Tabla 10).
- El pago y su verificación son manuales (fuera de alcance, supuesto S5).

6. Acceso seguro a la solución (criterio 4.1.5)

6.1 Librerías de repositorios oficiales y criterios de calidad

Para autenticación y protección de datos no se implementó criptografía propia: se usaron librerías publicadas en el repositorio oficial de Python (PyPI) y la biblioteca estándar. Cada recomendación de IA sobre seguridad se contrastó con los criterios técnicos de la Tabla 21; los casos que hubo que corregir después se registran en la Tabla 26. Las librerías utilizadas se listan en la Tabla 20.

Tabla 20. Librerías de repositorios oficiales utilizadas.

Librería	Repositorio	Versión mínima (requirements.txt)	Uso en el sistema
argon2-cffi	PyPI (pypi.org/project/argon2-cffi)	23.1.0	Hash de contraseñas con Argon2id y sal única (RNF-01).
cryptography (Fernet)	PyPI (pypi.org/project/cryptography)	42.0.0	Cifrado autenticado de RUT y teléfono antes de guardarlos (RNF-02).
fastapi	PyPI (pypi.org/project/fastapi)	0.115.0	Interfaz web: rutas, validación del formato de cada petición y cookie de sesión (RNF-12).
uvicorn	PyPI (pypi.org/project/uvicorn)	0.30.0	Servidor local que ejecuta la interfaz web.
pytest y httpx	PyPI	8.0 y 0.27.0	Solo para las pruebas automatizadas; httpx permite probar la interfaz web sin levantar un servidor.
secrets	Biblioteca estándar de Python	—	Identificador aleatorio de 256 bits para cada sesión web.
sqlite3	Biblioteca estándar de Python	—	Consultas parametrizadas (RNF-03).
hashlib y hmac	Biblioteca estándar de Python	—	Verificación de contraseñas guardadas con el algoritmo anterior (PBKDF2), que luego se migran a Argon2id.

Tabla 21. Criterios técnicos de calidad para validar librerías y recomendaciones de IA.

Criterio	Cómo se verificó	Resultado
Origen oficial	El paquete existe en PyPI con el nombre exacto y mantenedores reconocidos (previene paquetes falsos sugeridos por IA).	Cumple
Estándar reconocido	El algoritmo aparece en guías de referencia (OWASP Password Storage Cheat Sheet recomienda Argon2id; Fernet es la receta de alto nivel de cryptography).	Cumple
Mantenimiento activo	Versiones recientes publicadas y documentación oficial vigente.	Cumple
Sin vulnerabilidades conocidas	Ejecución de pip-audit sobre requirements.txt: “No known vulnerabilities found”.	Cumple
API difícil de usar mal	Parámetros seguros por defecto (Argon2id, sal automática, autenticación del mensaje en Fernet).	Cumple
Comprensión del equipo	Cada integrante puede explicar qué hace cada llamada (requisito de la guía).	Cumple

6.2 Mecanismo de autenticación

El flujo (BPMN 2) es: buscar al usuario con una consulta parametrizada → si no existe, verificar la clave contra un hash señuelo y responder con el mensaje genérico → revisar si la cuenta está bloqueada → verificar el hash Argon2id → registrar el intento fallido o reiniciar el contador → abrir una sesión que expira tras 30 minutos de inactividad. El mensaje de rechazo es siempre el mismo («Correo o contraseña incorrectos»), también para una cuenta bloqueada: en el BPMN 2 el bloqueo aparece como un final separado porque el sistema lo resuelve antes de verificar la clave, pero el usuario no puede distinguirlo. En la interfaz web la sesión se identifica con un valor aleatorio de 256 bits (secrets.token_hex) que viaja en una cookie HttpOnly y SameSite=Strict; una petición sin cookie recibe 401, y las peticiones que modifican datos deben traer además la cabecera X-Requested-With o reciben 403. Toda operación posterior comprueba la sesión y el permiso del rol dentro del servicio (exigir_rol y exigir_permiso_reserva), no en la interfaz, por lo que la consola y la web quedan protegidas por el mismo código.

6.3 Validación rigurosa de credenciales (Indicador 4.1.5.G.18)

La Tabla 22 describe cómo responde el sistema en cada escenario de acceso y qué prueba lo comprueba.

Tabla 22. Validación de credenciales por escenario.

Escenario	Comportamiento del sistema	Prueba
Correo inexistente	Mismo mensaje que una clave incorrecta; se calcula igualmente un hash (señuelo) para no revelar qué correos están registrados.	test_credenciales_invalidas_siempre_tienen_mensaje_generico
Contraseña incorrecta	Mensaje genérico y conteo del intento fallido.	test_credenciales_invalidas_siempre_tienen_mensaje_generico
5 intentos fallidos	Bloqueo de 15 minutos aun con la clave correcta, con el mismo mensaje genérico; se libera al pasar el tiempo.	test_bloquea_al_quinto_fallo_y_permite_login_a_los_15_minutos test_correo_inexistente_y_cuenta_bloqueada_comparten_mensaje
Contraseña débil al registrarse	Se exigen 10 a 128 caracteres con mayúscula, minúscula, número y símbolo.	test_politica_de_contrasena_tiene_limite_superior test_validadores_de_rut_mod11_y_password_con_caracter_especial
RUT inválido o correo repetido al registrarse	Se rechaza el registro (en la web, con los códigos 422 y 409).	test_registro_exige_rut_mod11_y_correo_unico test_responde_409_para_correo_duplicado test_responde_422_para_rut_o_password_invalidos
Recuperación de contraseña con RUT incorrecto	Mensaje único; cinco fallos bloquean la cuenta igual que en el inicio de sesión.	test_recuperacion_se_bloquea_tras_cinco_rut_incorrectos
Sesión expirada	Se exige iniciar sesión nuevamente.	test_sesion_expira_tras_30_minutos_inactiva test_rol_no_pueden_invocar_operaciones_de_paquetes_ajenas
Cliente intenta una operación de administrador	Rechazo de autorización.	test_cliente_no_puede_gestionar_destinos test_rol_no_pueden_invocar_operaciones_de_paquetes_ajenas
Cliente intenta cancelar una reserva ajena	Responde «No se encontró una reserva propia», sin confirmar que la reserva existe.	test_historial_aislado_por_usuario_y_cancelacion_propia
Petición web sin cookie o sin la cabecera	Responde 401 o 403 sin ejecutar la operación.	Comprobación manual (RNF-12); sin prueba automatizada

6.4 Protección de la información sensible (Indicador 4.1.5.I.19)

La Tabla 23 indica, para cada dato sensible, el mecanismo que lo protege y dónde se aplica.

Tabla 23. Protección de información sensible.

Dato	Mecanismo	Dónde y cómo
Contraseña	Hash Argon2id con sal única (no reversible)	Columna usuarios.password_hash; nunca se guarda, registra ni muestra la clave original (R10). Las contraseñas guardadas con el algoritmo anterior se migran a Argon2id cuando el usuario inicia sesión.
RUT y teléfono	Cifrado autenticado Fernet (AES-128-CBC + HMAC-SHA256)	Columnas usuarios.rut y usuarios.telefono, que guardan el texto cifrado; se descifran solo en memoria al cargar al cliente.
Visualización	Enmascarado	mask_rut y mask_phone (seguridad/enmascarado.py) muestran solo los últimos caracteres; ningún listado ni mensaje de error incluye el dato completo (R17).
Clave de cifrado	Variable de entorno VIAJES_FERNET_KEY	La clave no está en el código fuente, en el repositorio ni en la base de datos; sin ella el sistema no inicia. Si se pierde, los datos cifrados no pueden recuperarse (se indica en el README).
Registro (log)	Filtro FiltroDatosSensibles	Todo lo que se escribe en app.log pasa por un filtro que enmascara RUT y teléfonos y borra contraseñas; el archivo está excluido del repositorio.
Errores	Mensajes controlados	Las excepciones propias (excepciones.py) tienen textos seguros; los errores inesperados se registran y al usuario se le informa solo un mensaje genérico.

6.5 Análisis de amenazas y controles

La Tabla 24 relaciona cada amenaza con el control implementado y el riesgo que permanece.

Tabla 24. Amenazas, controles y riesgo residual.

Amenaza (categoría)	Control implementado	Riesgo residual
Fuerza bruta o adivinación de claves (Spoofing)	Argon2id (costoso de atacar), bloqueo de 15 minutos tras 5 fallos y política de contraseñas	Bajo; no hay segundo factor
Enumeración de usuarios	Mensaje único y hash señuelo en el inicio de sesión; mensaje único en la recuperación	Bajo
Recuperación de la contraseña por un tercero	Exige correo y RUT, y comparte el bloqueo por intentos fallidos	Medio: el RUT no es un dato secreto; la solución definitiva es un enlace enviado al correo (fuera de alcance, supuesto S11)
Inyección SQL (Tampering)	Consultas parametrizadas, validación de entrada y restricciones en la base de datos	Bajo; se verificó por inspección, sin prueba automatizada
Petición falsificada desde otro sitio (CSRF)	Cookie SameSite=Strict y cabecera X-Requested-With obligatoria en las peticiones que modifican datos	Bajo
Código inyectado en la página (XSS)	La página arma su contenido con textContent y no usa innerHTML	Bajo
Robo de la cookie de sesión	Cookie HttpOnly que expira a los 30 minutos	Medio: sin HTTPS la cookie viaja sin cifrar; hoy la web se usa solo en el mismo equipo

Amenaza (categoría)	Control implementado	Riesgo residual
Robo del archivo de base de datos (Information Disclosure)	Contraseñas con hash y RUT y teléfono cifrados; la clave está solo en una variable de entorno	Medio: quien acceda al equipo y a la variable puede descifrar los datos
Acceso a datos ajenos (Elevation of Privilege)	Autorización por rol en cada servicio y filtro por usuario en las consultas de reservas	Bajo
Sobreventa o reserva vencida (Tampering)	Transacción BEGIN IMMEDIATE, triggers de cupo y validación de la fecha	Bajo
Fuga por mensajes o registros (Information Disclosure)	Mensajes controlados, filtro del registro y app.log fuera del repositorio	Bajo
Negación de responsabilidad (Repudiation)	Cada reserva guarda fecha y cliente; los errores técnicos quedan en app.log	Medio: no se registran los inicios de sesión ni quién hizo cada cambio del catálogo
Dependencias vulnerables	Solo PyPI oficial y pip-audit sin hallazgos	Bajo; las versiones se declaran como mínimas, por lo que conviene repetir pip-audit en cada entrega

6.6 Evaluación y optimización de la seguridad con apoyo de IA (Indicador 4.1.5.I.20)

La evaluación se hizo revisando el código y el repositorio contra la lista de amenazas anterior, con apoyo de una herramienta de IA. Cada hallazgo se contrastó con el código y, cuando fue posible, con una prueba automatizada antes de dar por aplicada la mejora (Tabla 25):

Tabla 25. Hallazgos de seguridad y mejoras aplicadas.

Hallazgo	Mejora aplicada	Verificación
Responder distinto para «correo no existe» y «clave incorrecta» permite descubrir cuentas	Mensaje único y hash señuelo	test_credenciales_invalidas_siempre_tienen_mensaje_generico
Sin límite de intentos, un atacante puede probar claves indefinidamente	Bloqueo de 15 minutos tras 5 fallos	test_bloquea_al_quinto_fallo_y_permite_login_a_los_15_minutos
La recuperación de contraseña no tenía límite de intentos y respondía distinto si la cuenta era de un administrador	Bloqueo compartido con el inicio de sesión y mensaje único	test_recuperacion_se_bloquea_tras_cinco_rut_incorrectos
Contar el cupo y guardar la reserva por separado permitía sobreventa con dos reservas simultáneas	Transacción BEGIN IMMEDIATE y trigger de cupo	test_cupo_no_se_sobrecarga_con_reservas_concurrentes
Reservas duplicadas por reintentos o por dos socios	Triggers que impiden una segunda reserva activa del mismo cliente	test_impide_duplicado_activo_y_permite_reservar_tras_cancelacion
El README del repositorio público mostraba correos de usuarios reales, y se habían subido el registro app.log y archivos temporales	Se eliminaron los correos y se excluyeron esos archivos con .gitignore	Revisión del repositorio tras los cambios
Contraseñas de versiones anteriores guardadas con un algoritmo más débil (PBKDF2)	Migración automática a Argon2id al iniciar sesión	test_login_migra_hash_pbkdf2_a_argon2id
RUT y teléfono de versiones anteriores guardados en texto claro	Se cifran al abrir la base de datos	test_migra_rut_y_telefono_heredados_en_texto_claro
Posibles dependencias con vulnerabilidades	Auditoría con pip-audit sobre requirements.txt	Sin vulnerabilidades conocidas (5 de octubre de 2026)

6.7 Riesgos residuales y recomendaciones

- Guardar la clave de cifrado en un gestor de secretos, separada de la base de datos, y definir un procedimiento de respaldo y rotación (por ejemplo, MultiFernet).
- Reemplazar la recuperación con correo y RUT por un enlace de un solo uso enviado al correo, y agregar un segundo factor de autenticación, cuando exista envío de correos (hoy fuera de alcance).
- Antes de publicar la interfaz web: usar HTTPS, marcar la cookie como Secure y guardar las sesiones fuera de la memoria del servidor; si se migra a PostgreSQL, usar un usuario con mínimos privilegios.
- Agregar pruebas automatizadas para la inyección SQL (RNF-03) y para las respuestas 401 y 403 de la web (RNF-12), y registrar los inicios de sesión.
- Repetir pip-audit y actualizar las dependencias en cada entrega.

7. Uso crítico de herramientas de inteligencia artificial

La guía permite y espera el uso de IA como apoyo en la codificación, con tres condiciones: el equipo comprende el código, lo valida técnicamente y justifica las decisiones de uso, modificación o descarte. Por eso la IA se trató como un asistente que propone, y al equipo le correspondió decidir. Los casos en que una propuesta se aceptó sin la revisión suficiente y hubo que corregirla después se registran en la Tabla 26. Se usaron dos asistentes: Gemini CLI, configurado con las reglas del archivo gemini.md del repositorio, y Claude. Cada pedido, la respuesta obtenida y las pruebas ejecutadas quedaron registrados en el archivo changelog.md.

7.1 Proceso de revisión aplicado

1. **Proponer:** la IA sugiere una alternativa (algoritmo, estructura, consulta).
2. **Verificar en fuente oficial:** documentación de Python, PyPI, OWASP; se confirma que la librería y sus parámetros existen.
3. **Probar:** se escribe o ejecuta una prueba automatizada que demuestre el comportamiento esperado.
4. **Decidir y registrar:** aceptar, modificar o descartar, anotando el motivo (tabla siguiente).

7.2 Registro de contribuciones, análisis y decisiones

La Tabla 26 registra ocho contribuciones de las herramientas de IA, con la decisión del equipo y su fundamento.

Tabla 26. Registro de contribuciones de IA, análisis y decisiones.

N.º	Qué se pidió	Qué devolvió la IA	Decisión	Fundamento y verificación
1	Revisar el código y dejarlo sin errores de ejecución.	Corrigió sangrías con tabuladores, comillas de documentación mal cerradas y una importación duplicada en nueve archivos.	Aceptada	Eran cambios de forma que no tocaban las reglas; las 44 pruebas de ese momento siguieron aprobando.
2	Agregar una interfaz web reutilizando los servicios, sin cambiar su lógica.	api.py con FastAPI, sesión en cookie HttpOnly y SameSite=Strict, cabecera contra CSRF y una página que no usa innerHTML.	Aceptada	La web llama a los mismos servicios que la consola, por lo que no duplicó reglas; se comprobó que sin sesión responde 401. Los riesgos nuevos se declaran en la Tabla 24.
3	Permitir que un cliente recupere su contraseña.	Un método que cambia la clave con correo y RUT, sin límite de intentos y con un mensaje distinto para cuentas de administrador.	Modificada	El RUT no es secreto y sin límite se podía adivinar. Se agregó el bloqueo compartido con el inicio de sesión y un mensaje único; prueba <code>test_recuperacion_se_bloquea_tras_cinco_rut_incorrectos</code> .
4	Poder modificar el cupo y las fechas de un paquete ya creado.	Una edición que sobrescribía las fechas del paquete y recalculaba el precio si cambiaba el margen.	Modificada	Sobrescribir las fechas afectaba a las reservas ya hechas y recalculaba el precio contradice R7. Se reemplazó por salidas independientes (tabla paquete_salidas) y la web rechaza cambiar el margen; prueba <code>test_agregar_salida_conserva_la_salida_anterior</code> .
5	Corregir el error «Method Not Allowed» al agregar una salida.	Atribuyó el error dos veces a un servidor antiguo que seguía en ejecución.	Descartada	El error persistió. La traza del servidor mostró la causa real: una columna (salida_id) que faltaba al abrir bases ya existentes. Se corrigió el orden de creación del esquema.
6	Cumplir la rúbrica en herencia y polimorfismo.	La jerarquía Usuario, Cliente y Administrador, el hash señuelo y la política de contraseñas.	Modificada	La herencia existía, pero el servicio de reservas seguía comparando el texto del rol y Cliente repetía atributos heredados. Se agregó <code>exigir_permiso_reserva</code> , que usa <code>puede_reservar()</code> , y se simplificó Cliente.
7	Dejar en el README los usuarios registrados.	Una tabla con los nombres y correos reales de los usuarios.	Descartada	Eran datos personales en un repositorio público, contra R17. Se eliminaron del README.
8	Una matriz de trazabilidad de la rúbrica.	Una lista de archivos por tema, sin indicar qué prueba verifica cada regla.	Modificada	Se rehizo con una fila por regla y la prueba que la verifica, comprobando cada nombre contra el repositorio; así se detectó que R4 no tiene prueba dedicada.

7.3 Riesgos del uso de IA y cómo se controlaron

- Código que «parece correcto» pero falla en casos límite: se cubrió con 49 pruebas automatizadas, que se ejecutan después de cada cambio.
- Diagnósticos equivocados: en el caso 5 de la Tabla 26 la IA atribuyó dos veces el error a una causa falsa; se resolvió leyendo la traza del servidor.
- Documentación que no coincide con el código: la primera versión de este informe describía clases y pruebas que el repositorio no tenía; se corrigió contrastando cada nombre con el código (secciones 3, 5 y 6).

- Prácticas inseguras (datos personales en el README, recuperación de contraseña sin límite): se revisaron contra las reglas del caso y se corrigieron.
- Librerías o funciones inexistentes: se verificó cada dependencia en PyPI y en su documentación oficial antes de agregarla.
- Dependencia excesiva: cada integrante debe poder explicar y modificar los módulos que presenta en su defensa.

8. Conclusiones

La solución convierte las reglas y fallas del caso en un sistema verificable: el sobrecupo se evita con un cupo disponible calculado dentro de una transacción; las reservas vencidas se rechazan comparando con la fecha del día; el precio se calcula automáticamente y queda congelado, de modo que cambiar el costo de un destino no altera lo ya publicado ni lo ya cobrado; los destinos que dejan de operarse se desactivan sin romper los paquetes vendidos; y los datos sensibles dejan de circular en planillas para quedar cifrados y restringidos por rol.

Desde lo metodológico, el trabajo mostró que separar regla de negocio y requerimiento ordena el análisis, y que la trazabilidad (requerimiento → caso de uso → BPMN → clase → tabla → prueba) permite demostrar, y no solo afirmar, que el sistema responde al problema del cliente. Desde lo técnico, reforzar las reglas también en la base de datos y usar librerías oficiales de seguridad redujo el riesgo sin aumentar la complejidad.

Trabajo futuro: publicar la interfaz web con HTTPS, segundo factor de autenticación, recuperación de contraseña y notificaciones por correo, informes de gestión para los socios, verificación automática de pagos y migración a un motor de base de datos cliente-servidor.

Qué se haría distinto: escribir el informe a partir del código y no antes, porque la primera versión describía un sistema que todavía no existía; integrar el trabajo de todos desde el primer día, con cambios pequeños y frecuentes en el repositorio; definir una sola interfaz antes de empezar, porque mantener consola y web dejó operaciones disponibles solo en una de ellas; y escribir la prueba de cada regla junto con la regla.

Referencias

Cuevas Álvarez, A. (2016). *Python 3*. RA-MA Editorial.

Jiménez de Parga, C. (2021). *UML: arquitectura de aplicaciones en Java, C++ y Python* (1.^a ed.). Ra-Ma.

Sánchez Palacio, A. (2025). *ChatGPT y OpenAI: desarrollo y uso de herramientas de inteligencia artificial generativa*. RA-MA Editorial.

Object Management Group. (2011). *Business Process Model and Notation (BPMN), versión 2.0*. <https://www.omg.org/spec/BPMN/2.0/>

OWASP Foundation. (s. f.). *Password Storage Cheat Sheet y Authentication Cheat Sheet*. <https://cheatsheetseries.owasp.org/>

Python Software Foundation. (s. f.). *sqlite3: DB-API 2.0 interface for SQLite databases y secrets: generate secure random numbers*. <https://docs.python.org/3/>

argon2-cffi. (s. f.). *Documentación oficial y repositorio del paquete en PyPI*. <https://argon2-cffi.readthedocs.io/>

Python Cryptographic Authority. (s. f.). *cryptography: Fernet (symmetric encryption)*.
<https://cryptography.io/>

Schwaber, K. y Sutherland, J. (2020). *La Guía de Scrum*. <https://scrumguides.org/>